

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Thực tập đồ án Đa ngành - Hướng Trí tuệ Nhân tạo (CO3107)

Project **YOLO:HOME**

Nhóm: 14 - L07

Giảng viên hướng dẫn: Lê Bình Đăng
Sinh viên thực hiện: Đồng Mạnh Cường - 2210420
Phan Minh Trọng - 2213676
Hồ Quang Long - 2211878
Đỗ Tuấn Đạt - 2210667
Đậu Ngọc Quân - 2212777

TP. Hồ Chí Minh, 2025



Mục lục

1	Giới thiệu	3
1.1	Tầm nhìn của YOLO:HOME	3
1.2	Vai trò của trí tuệ nhân tạo trong YOLO:HOME	4
1.3	Sự kết hợp đa ngành	4
1.4	Giá trị thực tiễn và giáo dục	4
2	Cơ sở lý thuyết	6
2.1	Các thiết bị chính	6
2.1.1	Remote và mắt đọc tín hiệu hồng ngoại	6
2.1.2	Cảm biến nhiệt độ độ ẩm DHT20	6
2.1.3	Cảm biến hồng ngoại	7
2.1.4	Cảm biến ánh sáng	7
2.1.5	Động cơ servo	8
2.1.6	Đèn 4 LED RGB	9
2.1.7	Màn hình LCD	9
2.1.8	Quạt mini	10
2.1.9	Yolo: Bit	11
2.2	Phần mềm	11
2.2.1	ReactJS cho giao diện người dùng (Frontend)	11
2.2.2	Node.js và Express.js cho phía server (Backend)	12
2.2.3	MongoDB cho cơ sở dữ liệu	13
2.2.4	Figma cho thiết kế giao diện	14
2.3	Phần trí tuệ nhân tạo	14
2.3.1	Google Speech-to-Text API	14
2.3.2	Mô hình SSD Caffe và ArcFace	16
3	Phân tích	19
3.1	Hệ thống phần cứng	19
3.1.1	Giám sát môi trường	19
3.2	Phần mềm	20
3.2.1	Yêu cầu người dùng	20
3.2.2	Use Case Detail	22
3.2.3	Use-case diagram	35
3.3	Phần trí tuệ nhân tạo	36
3.3.1	Nhận diện giọng nói	36
3.3.2	Nhận diện khuôn mặt	37
4	Thiết kế	39
4.1	Deployment View	39
4.2	Hệ thống phần cứng	40
4.3	Phần mềm	41
4.3.1	Thiết kế UI/UX	41
4.3.2	Database	47
4.4	Phần trí tuệ nhân tạo	53
4.4.1	Nhận diện giọng nói	53
4.4.2	Nhận diện khuôn mặt	54
5	Hiện thực	56



5.1	Hệ thống phần cứng	56
5.1.1	Màn hình LCD1602, DHT20 và Thời gian	56
5.1.2	Điều khiển các thiết bị thủ công	57
5.1.3	Tự động bật/tắt các thiết bị	58
5.2	Phần mềm	60
5.3	Phần trí tuệ nhân tạo	64
5.3.1	Nhận diện giọng nói	64
5.3.2	Nhận diện khuôn mặt	66
6	Tổng kết	69
6.1	Những gì đã làm được	69
6.2	Hướng cải thiện trong tương lai	69

1 Giới thiệu

Trong bối cảnh cuộc cách mạng công nghiệp 4.0, trí tuệ nhân tạo (AI) đã trở thành động lực cốt lõi thúc đẩy sự phát triển của các lĩnh vực công nghệ, từ tự động hóa đến các hệ thống tự hành thông minh. Kết hợp với Internet vạn vật (IoT), AI tạo nên khái niệm **AIoT** (Artificial Intelligence of Things), mở ra tiềm năng to lớn cho các giải pháp tích hợp, nơi các thiết bị không chỉ kết nối mà còn có khả năng học hỏi, phân tích, và đưa ra quyết định. Trong khuôn khổ **Đồ án Đa ngành - Hướng Trí tuệ nhân tạo (CO3107)** nhóm 14, , đã phát triển dự án **YOLO:HOME** – một hệ thống nhà thông minh tiên tiến, tận dụng sức mạnh của AIoT để mang lại trải nghiệm sống hiện đại, an toàn, và tiện nghi. Dự án không chỉ là minh chứng cho sự sáng tạo và năng lực kỹ thuật của nhóm mà còn thể hiện sự kết hợp đa ngành giữa trí tuệ nhân tạo, kỹ thuật phần cứng, và thiết kế phần mềm, đáp ứng các yêu cầu phức tạp của một thế giới công nghệ đang không ngừng thay đổi.

Đồ án Đa ngành - Hướng Trí tuệ nhân tạo (CO3107) là một học phần quan trọng, được thiết kế để thử thách sinh viên trong việc tích hợp các kiến thức từ nhiều lĩnh vực, bao gồm lập trình, kỹ thuật điện tử, xử lý dữ liệu, và trí tuệ nhân tạo, để giải quyết các bài toán thực tiễn. Khác với các đồ án truyền thống, CO3107 yêu cầu sinh viên không chỉ tập trung vào một khía cạnh kỹ thuật mà còn phải phối hợp các lĩnh vực khác nhau để tạo ra một sản phẩm hoàn chỉnh, có khả năng ứng dụng thực tế. Trong bối cảnh đó, YOLO:HOME được xây dựng với mục tiêu không chỉ đáp ứng các yêu cầu học thuật mà còn đặt nền móng cho các giải pháp nhà thông minh mang tính đột phá, phù hợp với xu hướng công nghệ toàn cầu và nhu cầu của thị trường Việt Nam.

Dự án YOLO:HOME tận dụng nền tảng microcontroller **Yolo:Bit**, một thiết bị nhỏ gọn nhưng mạnh mẽ, hỗ trợ kết nối Wi-Fi và giao tiếp với server OhStem. Bằng cách kết hợp các kỹ thuật AI tiên tiến như nhận diện giọng nói (Speech-to-Text) và nhận diện khuôn mặt (FaceAI), cùng với khả năng điều khiển IoT, YOLO:HOME mang đến một hệ sinh thái thông minh, nơi các thiết bị không chỉ thực hiện các lệnh đơn giản mà còn có khả năng tự động hóa và đưa ra quyết định dựa trên dữ liệu môi trường và hành vi người dùng. Sự đa ngành được thể hiện rõ qua việc tích hợp phần cứng (cảm biến, servo, LED), phần mềm (giao diện người dùng, cơ sở dữ liệu), và các mô hình AI (Google Speech-to-Text, SSD, ArcFace), tạo nên một sản phẩm toàn diện, phản ánh đầy đủ tinh thần của đồ án.

1.1 Tầm nhìn của YOLO:HOME

YOLO:HOME không chỉ là một hệ thống nhà thông minh mà còn là một nền tảng AIoT tiên phong, được thiết kế để nâng cao chất lượng cuộc sống thông qua các tính năng thông minh và bảo mật. Với bốn module chính – giám sát môi trường, hệ thống cửa mật mã, điều khiển bằng giọng nói, và nhận diện khuôn mặt (FaceAI) – dự án mang đến một giải pháp toàn diện cho các nhu cầu quản lý không gian sống. Mỗi module đều được phát triển với sự tập trung đặc biệt vào trí tuệ nhân tạo, từ việc sử dụng Google Speech-to-Text API để nhận diện giọng nói tiếng Việt, đến việc triển khai các mô hình SSD và ArcFace cho nhận diện khuôn mặt, đảm bảo tính chính xác và hiệu quả.

- **Giám sát môi trường:** Sử dụng các cảm biến như DHT20 (nhiệt độ, độ ẩm), cảm biến ánh sáng, và hồng ngoại, hệ thống thu thập dữ liệu thời gian thực, xử lý và hiển thị dưới dạng biểu đồ hoặc bảng. AI được ứng dụng để phân tích dữ liệu, dự đoán xu hướng, và tự động kích hoạt các thiết bị như quạt hoặc máy phun sương khi môi trường vượt ngưỡng an toàn.

- **Hệ thống cửa mật mã:** Đảm bảo an ninh với cơ chế nhập mật mã qua remote, hỗ trợ thay đổi mật mã, giới hạn số lần nhập sai, và ghi lại lịch sử truy cập. AI hỗ trợ trong việc phân tích các mẫu truy cập để phát hiện hành vi bất thường.
- **Điều khiển bằng giọng nói:** Tận dụng Google Speech-to-Text API, YOLO:HOME cho phép người dùng điều khiển các thiết bị như đèn LED RGB hoặc quạt bằng giọng nói tiếng Việt. Hệ thống lưu trữ lịch sử lệnh và báo lỗi khi không nhận diện được, mang lại trải nghiệm tương tác tự nhiên.
- **Nhận diện khuôn mặt (FaceAI):** Sử dụng mô hình SSD để phát hiện khuôn mặt và ArcFace để tạo embedding phân tách, hệ thống xác thực danh tính để mở cửa, gửi cảnh báo khi phát hiện khuôn mặt lạ, và lưu nhật ký truy cập. Đây là tính năng nổi bật, thể hiện rõ sự ứng dụng sâu rộng của AI trong bảo mật.

1.2 Vai trò của trí tuệ nhân tạo trong YOLO:HOME

Trí tuệ nhân tạo là linh hồn của YOLO:HOME, đóng vai trò trung tâm trong việc biến hệ thống từ một giải pháp tự động hóa đơn thuần thành một nền tảng tự hành thông minh. Các mô hình AI được triển khai không chỉ đảm bảo tính chính xác mà còn tăng cường khả năng tương tác và bảo mật của hệ thống. Ví dụ, Google Speech-to-Text API cho phép nhận diện giọng nói tiếng Việt với độ chính xác cao, phù hợp với người dùng tại Việt Nam. Trong khi đó, mô hình SSD và ArcFace mang lại khả năng nhận diện khuôn mặt nhanh chóng và đáng tin cậy, ngay cả trong điều kiện ánh sáng không lý tưởng. Những công nghệ này không chỉ đáp ứng yêu cầu kỹ thuật của đồ án mà còn mở ra tiềm năng ứng dụng trong các lĩnh vực như an ninh, quản lý tòa nhà, và chăm sóc sức khỏe.

Sự tích hợp AI còn thể hiện qua khả năng học hỏi và tối ưu hóa. Hệ thống có thể phân tích dữ liệu môi trường để đưa ra các quyết định tự động, như bật quạt khi nhiệt độ cao, hoặc gửi cảnh báo khi phát hiện bất thường. Trong tương lai, nhóm dự kiến cải tiến bằng cách sử dụng các mô hình AI cục bộ như OpenAI Whisper để giảm phụ thuộc vào dịch vụ đám mây, hoặc fine-tune ArcFace để tăng độ chính xác trong nhận diện khuôn mặt. Những nỗ lực này phản ánh tinh thần đổi mới và cam kết của nhóm trong việc đẩy nhanh quá trình ứng dụng AI vào thực tiễn.

1.3 Sự kết hợp đa ngành

YOLO:HOME là kết quả của sự phối hợp chặt chẽ giữa các lĩnh vực kỹ thuật. Về phần cứng, nhóm đã thiết kế và tích hợp các cảm biến, động cơ servo, và màn hình LCD với Yolo:Bit, đảm bảo khả năng hoạt động ổn định. Về phần mềm, hệ thống sử dụng cơ sở dữ liệu MongoDB với các schema như `doorSchema`, `EnvironmentDataSchema`, và `FaceDataSchema` để quản lý thông tin hiệu quả. Giao diện người dùng (UI/UX) được thiết kế trên Figma, bao gồm các trang đăng nhập, giám sát môi trường, và điều khiển thiết bị, mang lại trải nghiệm trực quan và thân thiện. Đặc biệt, sự kết hợp giữa các mô hình AI và kiến trúc client-server đã tạo nên một hệ thống linh hoạt, có thể mở rộng để hỗ trợ nhiều thiết bị và tính năng hơn trong tương lai.

1.4 Giá trị thực tiễn và giáo dục

YOLO:HOME không chỉ là một dự án học thuật mà còn có tiềm năng ứng dụng rộng rãi trong thực tế. Hệ thống có thể được triển khai trong các hộ gia đình, văn phòng, hoặc khu công nghiệp để nâng cao an ninh, tiết kiệm năng lượng, và cải thiện chất lượng cuộc sống. Đặc biệt, với khả



năng hỗ trợ tiếng Việt và tích hợp các cảm biến giá thành thấp, YOLO:HOME rất phù hợp với thị trường Việt Nam, nơi các giải pháp nhà thông minh đang ngày càng được ưa chuộng.

Về mặt giáo dục, dự án là một tài liệu tham khảo quý giá cho sinh viên và nhà nghiên cứu trong lĩnh vực AIoT. Các sơ đồ hoạt động, thiết kế cơ sở dữ liệu, và giao diện UI/UX được trình bày trong báo cáo có thể truyền cảm hứng cho các dự án tương lai. Đồng thời, quá trình phát triển YOLO:HOME đã giúp nhóm rèn luyện các kỹ năng quan trọng như làm việc nhóm, giải quyết vấn đề, và quản lý dự án, chuẩn bị sẵn sàng cho các thách thức trong ngành công nghệ.

YOLO:HOME là bước khởi đầu cho hành trình khám phá tiềm năng của AIoT. Trong tương lai, nhóm dự kiến mở rộng hệ thống để hỗ trợ thêm các thiết bị thông minh, tích hợp các mô hình AI tiên tiến hơn, và phát triển ứng dụng di động để quản lý từ xa. Việc triển khai các mô hình AI cục bộ và tối ưu hóa hiệu suất cũng là mục tiêu dài hạn, nhằm giảm chi phí và tăng tính độc lập của hệ thống.

2 Cơ sở lý thuyết

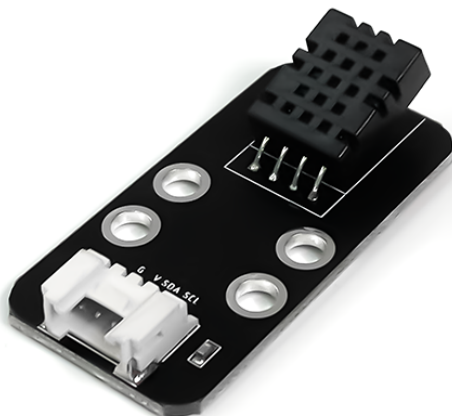
2.1 Các thiết bị chính

2.1.1 Remote và mắt đọc tín hiệu hồng ngoại



Remote IR (đóng vai trò là bên gửi) sẽ phát ra một số các tín hiệu hồng ngoại tương trưng cho các tín hiệu mã nhị phân. Các tín hiệu này sẽ tạo thành một chuỗi mã nhị phân có ý nghĩa, chẳng hạn như khi ta nhấn phím A hoặc F trên Remote, Remote sẽ phát ra tia hồng ngoại tương ứng với các phím đó và khi mắt IR (đóng vai trò là bên nhận) sẽ nhận được các tín hiệu đó. Các tín hiệu sau đó được chuyển cho Vi điều khiển (micro-controller) và Vi điều khiển sẽ xử lý các tín hiệu theo các chức năng của người dùng.

2.1.2 Cảm biến nhiệt độ độ ẩm DHT20



DHT20 là một cảm biến nhiệt độ độ ẩm giao tiếp với các Vi điều khiển bằng giao thức I2C (Inter-Integrated Circuit). DHT20 sử dụng cảm biến độ ẩm điện dung để xác định độ ẩm của không khí và nhiệt điện trở để xác định nhiệt độ môi trường. Sau đó, khi đã có được dữ liệu cần thiết, DHT20 sẽ giao tiếp với Vi điều khiển bằng giao thức I2C để gửi các dữ liệu.

2.1.3 Cảm biến hồng ngoại



Cảm biến PIR (hồng ngoại thụ động) sử dụng khả năng phát hiện tia hồng ngoại phát ra từ mọi vật thể phát ra nhiệt. Tia hồng ngoại này không thể nhìn thấy bằng mắt thường, nhưng các cảm biến hoạt động bằng bước sóng hồng ngoại có thể phát hiện các loại tia đó. Chúng thường có tên gọi khác là cảm biến chuyển động bởi vì chúng có thể phát hiện ra sự hiện diện của một người nào đó, hay một loài vật nào đó dựa trên các tia hồng ngoại được phát ra.

2.1.4 Cảm biến ánh sáng



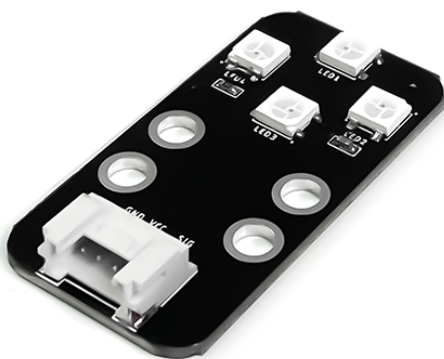
Cảm biến ánh sáng dựa trên hoạt động chính của quang trở. Quang trở là một loại điện trở thay đổi giá trị điện trở khi có ánh sáng chiếu trực tiếp vào nó. Giá trị của điện trở sẽ giảm khi cường độ ánh sáng chiếu trực tiếp vào nó thấp và sẽ tăng khi cường độ ánh sáng chiếu trực tiếp vào nó cao.

2.1.5 Động cơ servo



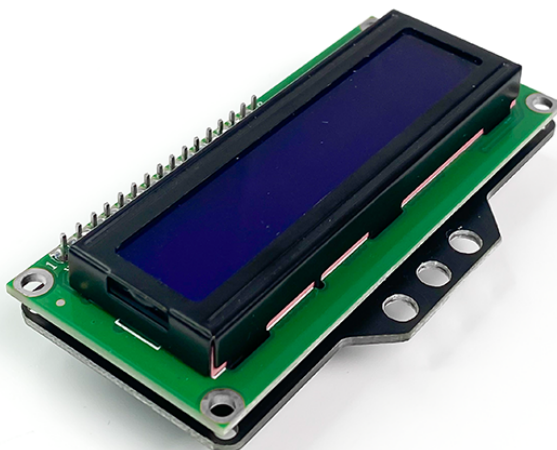
Động cơ Servo SG90 sẽ đóng vai trò là cửa chính của ngôi nhà. Cách thức hoạt động của động cơ SG90 dựa trên các xung PWM của Vi điều khiển gửi đến SG90. Trong SG90 sẽ có một loại Vi xử lý nhận các tín hiệu từ Vi điều khiển và điều khiển động cơ Servo quay theo góc dựa trên Duty Cycle của xung PWM.

2.1.6 Đèn 4 LED RGB



Module 4 LED RGB là gồm có 4 đèn LED RGB ws2812 đủ màu. Với chip tích hợp có thể điều khiển từng đèn LED riêng lẻ hoặc tắt cả đèn LED có trên nó. Ngoài ra, module cũng có thể điều chỉnh độ sáng và tạo ra bất kỳ màu nào trong bảng màu RGB.

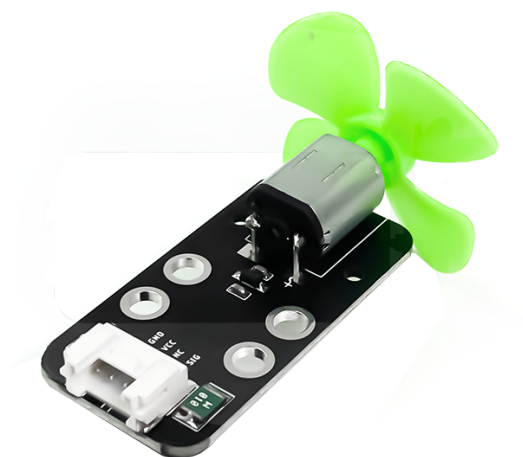
2.1.7 Màn hình LCD



Màn hình LCD 1602 là thiết bị điện tử có thể hiển thị 2 dòng, mỗi dòng 16 ký tự. LCD 1602 có thể giao tiếp với các Vi điều khiển để hiển thị bất kỳ ký tự, bất kỳ thông tin. Module này có độ

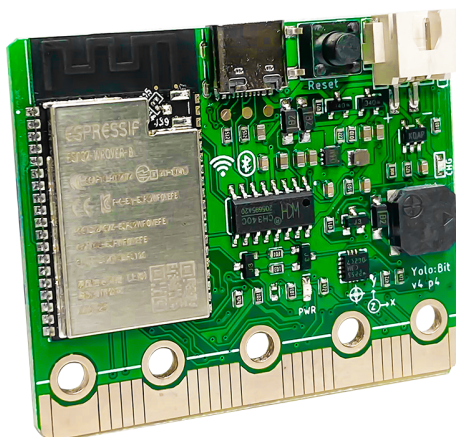
bền cao, đồng thời có tích hợp module giao tiếp I2C giúp việc giao tiếp được dễ dàng và nhanh chóng hơn rất nhiều.

2.1.8 Quạt mini



Động cơ quạt mini gồm 2 bộ phận: động cơ và cánh quạt có thể tạo thành một quạt máy mini và ứng dụng trong nhiều dự án sáng tạo khác nhau. Động cơ quạt có thể được điều chỉnh tốc độ theo nhu cầu.

2.1.9 Yolo: Bit



Yolo: Bit được tích hợp bộ Vi xử lý ESP32 WROVER B sẽ đóng vai trò là bộ não của ngôi nhà thông minh, đây là thiết bị quản lý, phân tích và xử lý các dữ liệu nhận được, đồng thời điều khiển các khối module khác. Mạch Yolo: Bit được tích hợp sẵn các tính năng cần thiết như kết nối Wifi, giao thức I2C và các chân kết nối GPIO (General Purpose Input Output) phù hợp để điều khiển các thiết bị ngoại vi và sử dụng hệ thống IoT.

2.2 Phần mềm

Để phát triển dự án **YOLO:HOME**, nhóm đã lựa chọn một hệ sinh thái các công cụ phần mềm hiện đại, phù hợp với yêu cầu của một hệ thống nhà thông minh tích hợp trí tuệ nhân tạo và Internet vạn vật (AIoT). Các công nghệ được sử dụng bao gồm **ReactJS** cho giao diện người dùng (frontend), **Node.js** và **Express.js** cho phía server (backend), **MongoDB** cho cơ sở dữ liệu, **Figma** cho thiết kế giao diện. Sự kết hợp này không chỉ đảm bảo hiệu suất và tính mở rộng của hệ thống mà còn mang lại trải nghiệm người dùng mượt mà, bảo mật và thân thiện. Dưới đây, mỗi công nghệ sẽ được trình bày chi tiết, bao gồm giới thiệu, tác dụng, và lý do nhóm lựa chọn.

2.2.1 ReactJS cho giao diện người dùng (Frontend)

ReactJS là một thư viện JavaScript mã nguồn mở, được phát triển bởi Facebook, chuyên dùng để xây dựng giao diện người dùng (UI) tương tác, đặc biệt là các ứng dụng web đơn trang (Single Page Applications - SPA). React hoạt động dựa trên mô hình component-based, cho phép các

nhà phát triển chia nhỏ giao diện thành các thành phần độc lập, có thể tái sử dụng và dễ dàng bảo trì. React sử dụng Virtual DOM (Document Object Model ảo) để tối ưu hóa việc cập nhật giao diện, giảm thời gian tải trang và nâng cao hiệu suất.

Trong dự án YOLO:HOME, ReactJS được sử dụng để xây dựng giao diện người dùng cho các trang như trang đăng nhập, trang giám sát môi trường (hiển thị dữ liệu nhiệt độ, độ ẩm, ánh sáng), và trang điều khiển thiết bị (đèn, quạt). Các component như biểu đồ dữ liệu môi trường (sử dụng thư viện Chart.js tích hợp với React) và bảng lịch sử truy cập được thiết kế để hiển thị thông tin theo thời gian thực, mang lại trải nghiệm trực quan và mượt mà. Ngoài ra, React Router được sử dụng để quản lý định tuyến giữa các trang, đảm bảo người dùng có thể chuyển đổi giữa các chức năng mà không cần tải lại toàn bộ trang.

Nhóm lựa chọn ReactJS vì các lý do sau:

- **Tính mô-đun và tái sử dụng:** Mô hình component-based giúp nhóm dễ dàng phát triển và bảo trì các thành phần giao diện riêng lẻ, như component biểu đồ hoặc nút điều khiển thiết bị.
- **Hiệu suất cao:** Virtual DOM giúp giảm thiểu các thao tác cập nhật trực tiếp trên DOM thực, cải thiện tốc độ hiển thị dữ liệu thời gian thực, đặc biệt quan trọng với các tính năng giám sát môi trường.
- **Cộng đồng hỗ trợ mạnh mẽ:** React có một hệ sinh thái phong phú với nhiều thư viện hỗ trợ như Redux (quản lý trạng thái), Axios (gửi yêu cầu HTTP), và Chart.js, giúp nhóm nhanh chóng tích hợp các tính năng phức tạp.
- **Phù hợp với nhóm:** Với kinh nghiệm lập trình JavaScript của các thành viên, React là lựa chọn tự nhiên, cho phép nhóm tập trung vào phát triển tính năng thay vì học một công nghệ hoàn toàn mới.

ReactJS đã giúp YOLO:HOME đạt được giao diện người dùng hiện đại, đáp ứng nhanh, và dễ dàng mở rộng để tích hợp thêm các tính năng trong tương lai, như điều khiển từ xa qua ứng dụng di động.

2.2.2 Node.js và Express.js cho phía server (Backend)

Node.js là một môi trường runtime JavaScript, cho phép chạy mã JavaScript bên ngoài trình duyệt, chủ yếu để xây dựng các ứng dụng phía server. Điểm nổi bật của Node.js là mô hình bất đồng bộ (asynchronous) dựa trên sự kiện (event-driven), giúp xử lý nhiều yêu cầu đồng thời mà không bị chặn, lý tưởng cho các ứng dụng yêu cầu thời gian thực như YOLO:HOME. Kết hợp với **Express.js**, một framework nhẹ của Node.js, nhóm đã xây dựng một server mạnh mẽ để xử lý các yêu cầu từ frontend, giao tiếp với cơ sở dữ liệu, và tích hợp các API bên thứ ba như Google Speech-to-Text.

Trong YOLO:HOME, server Node.js/Express.js đóng vai trò trung tâm, xử lý các tác vụ như:

- Quản lý yêu cầu điều khiển thiết bị (bật/tắt đèn, quạt) từ giao diện người dùng hoặc lệnh giọng nói.
- Gửi và nhận dữ liệu từ các cảm biến (nhiệt độ, độ ẩm, ánh sáng) thông qua giao tiếp với Yolo:Bit qua server OhStem.

- Xử lý logic xác thực (đăng nhập, nhận diện khuôn mặt) và lưu trữ lịch sử truy cập.
- Tích hợp các mô hình AI như SSD và ArcFace để xử lý dữ liệu nhận diện khuôn mặt.

Nhóm lựa chọn Node.js và Express.js vì:

- **Xử lý thời gian thực:** Mô hình bất đồng bộ của Node.js phù hợp với các tính năng thời gian thực của YOLO:HOME, như cập nhật dữ liệu cảm biến hoặc xử lý lệnh giọng nói.
- **Tích hợp dễ dàng với frontend:** Vì cả frontend (ReactJS) và backend (Node.js) đều sử dụng JavaScript, nhóm có thể duy trì một ngôn ngữ lập trình thống nhất, giảm thời gian học tập và tăng hiệu quả phát triển.
- **Khả năng mở rộng:** Express.js cung cấp một kiến trúc nhẹ, dễ dàng thêm middleware để xử lý các tác vụ như xác thực JWT, logging, hoặc tích hợp API bên thứ ba.
- **Hệ sinh thái phong phú:** Node.js có hàng ngàn gói npm (Node Package Manager), như `mongoose` để làm việc với MongoDB hoặc `axios` để gửi yêu cầu HTTP, giúp nhóm nhanh chóng xây dựng các tính năng phức tạp.

Sự kết hợp giữa Node.js và Express.js đã mang lại một backend ổn định, hiệu quả, và dễ dàng tích hợp với các thành phần khác của hệ thống YOLO:HOME.

2.2.3 MongoDB cho cơ sở dữ liệu

MongoDB là một cơ sở dữ liệu NoSQL dựa trên tài liệu (document-based), lưu trữ dữ liệu dưới dạng JSON-like (BSON). Khác với cơ sở dữ liệu quan hệ truyền thống, MongoDB không yêu cầu schema cố định, cho phép lưu trữ dữ liệu linh hoạt và dễ dàng mở rộng. Điều này đặc biệt hữu ích cho các ứng dụng như YOLO:HOME, nơi dữ liệu từ các nguồn khác nhau (cảm biến, lịch sử truy cập, nhận diện khuôn mặt) có cấu trúc đa dạng.

Trong YOLO:HOME, MongoDB được sử dụng để lưu trữ:

- Dữ liệu môi trường (nhiệt độ, độ ẩm, ánh sáng) với schema `EnvironmentDataSchema`.
- Lịch sử truy cập cửa với schema `doorSchema`, bao gồm thời gian, mật mã, và trạng thái (thành công/thất bại).
- Dữ liệu nhận diện khuôn mặt với schema `FaceDataSchema`, lưu trữ embedding khuôn mặt và thông tin người dùng.
- Lịch sử lệnh giọng nói để phân tích hành vi người dùng.

Nhóm sử dụng thư viện `mongoose` để tương tác với MongoDB từ Node.js, đảm bảo dữ liệu được truy vấn và cập nhật hiệu quả.

Lý do nhóm chọn MongoDB bao gồm:

- **Linh hoạt với dữ liệu không đồng nhất:** Dữ liệu từ cảm biến và nhận diện khuôn mặt có cấu trúc khác nhau, và MongoDB cho phép lưu trữ mà không cần định nghĩa schema cứng nhắc.

- **Hiệu suất cao:** MongoDB hỗ trợ truy vấn nhanh trên khối lượng dữ liệu lớn, phù hợp với các tính năng thời gian thực như giám sát môi trường.
- **Tích hợp tốt với Node.js:** Mongoose cung cấp một API trực quan để làm việc với MongoDB, giúp nhóm dễ dàng thực hiện các thao tác CRUD (Create, Read, Update, Delete).
- **Ả năng mở rộng**:** MongoDB hỗ trợ sharding và replication, đảm bảo hệ thống có thể xử lý khối lượng dữ liệu lớn hơn trong tương lai khi YOLO:HOME mở rộng.

MongoDB đã giúp nhóm xây dựng một hệ thống lưu trữ dữ liệu hiệu quả, đáp ứng yêu cầu của các tính năng AIoT trong YOLO:HOME.

2.2.4 Figma cho thiết kế giao diện

Figma là một công cụ thiết kế giao diện dựa trên đám mây, được sử dụng để tạo prototype, mockup, và thiết kế UI/UX. Figma hỗ trợ làm việc nhóm theo thời gian thực, cho phép nhiều thành viên cùng chỉnh sửa và đóng góp vào thiết kế. Trong YOLO:HOME, Figma được sử dụng để thiết kế giao diện cho các trang web, bao gồm bố cục, màu sắc, và các thành phần như nút, biểu đồ, và bảng dữ liệu.

Các thiết kế trên Figma bao gồm:

- Trang đăng nhập với giao diện đơn giản, tập trung vào tính bảo mật.
- Trang giám sát môi trường với các biểu đồ động (line chart, bar chart) để hiển thị dữ liệu cảm biến.
- Trang điều khiển thiết bị với các nút bật/tắt và hiển thị trạng thái thiết bị.

Nhóm chọn Figma vì:

- **Hỗ trợ làm việc nhóm:** Figma cho phép các thành viên cùng chỉnh sửa thiết kế, giúp nhóm nhanh chóng thống nhất ý tưởng.
- **Giao diện trực quan:** Figma cung cấp các công cụ thiết kế mạnh mẽ, dễ sử dụng, phù hợp với nhóm không có nhiều kinh nghiệm thiết kế đồ họa.
- **Tích hợp với phát triển:** Figma cho phép xuất mã CSS hoặc chia sẻ thiết kế trực tiếp với lập trình viên frontend, giảm thời gian chuyển đổi từ thiết kế sang code.

Figma đã giúp nhóm tạo ra giao diện người dùng thẩm mỹ, đồng nhất, và dễ dàng triển khai trên ReactJS.

2.3 Phần trí tuệ nhân tạo

2.3.1 Google Speech-to-Text API

Giới thiệu

Google Speech-to-Text là một dịch vụ mạnh mẽ của Google cho phép chuyển đổi giọng nói thành văn bản. Công nghệ này ứng dụng các kỹ thuật Machine Learning tiên tiến, đặc biệt là Deep Learning, để cải thiện độ chính xác và khả năng nhận diện ngôn ngữ.

Cấu trúc và kiến trúc của Google Speech-to-Text

- **Mô hình ngôn ngữ (Language Model):** Giúp nhận diện từ ngữ dựa trên ngữ cảnh và dự đoán các từ tiếp theo.
- **Mô hình âm học (Acoustic Model):** Phân tích các đặc trưng của âm thanh để nhận diện ký tự hoặc âm vị.
- **Mô hình giải mã (Decoder Model):** Tích hợp thông tin từ mô hình ngôn ngữ và âm học để đưa ra kết quả chính xác nhất.
- **Xử lý tín hiệu:** Google áp dụng các thuật toán DSP (Digital Signal Processing) để loại bỏ tạp âm và tối ưu chất lượng âm thanh.

Phương thức hoạt động

Google Speech-to-Text hoạt động dựa trên ba giai đoạn chính:

1. **Thu âm thanh:** Âm thanh được thu từ thiết bị micro và được chuẩn hóa thành dạng tín hiệu số.
2. **Xử lý tín hiệu:** Áp dụng các thuật toán DSP để loại bỏ tạp âm và tối ưu chất lượng âm thanh.
3. **Phân tích và nhận diện:** Sử dụng mô hình học sâu để phân tích các đặc trưng âm thanh và đối chiếu với kho từ vựng để chuyển thành văn bản.

Tính năng nổi bật

- **Hỗ trợ đa ngôn ngữ:** Google Speech-to-Text hỗ trợ hơn 125 ngôn ngữ và phương ngữ khác nhau.
- **Chế độ nhận diện trực tiếp và theo thời gian thực:** Có thể chuyển đổi giọng nói sang văn bản ngay khi người dùng nói.
- **Xử lý tiếng ồn vượt trội:** Công nghệ này có khả năng nhận diện tốt ngay cả trong môi trường có nhiều tiếng ồn.
- **Chuyển đổi văn bản có dấu câu:** Tự động thêm dấu chấm câu vào văn bản sau khi nhận diện.

Ứng dụng thực tế

Google Speech-to-Text được ứng dụng rộng rãi trong nhiều lĩnh vực:

- **Trợ lý ảo:** Giúp người dùng tương tác với trợ lý ảo như Google Assistant, Siri, Alexa.
- **Hệ thống thoại tự động (IVR):** Ứng dụng trong các tổng đài chăm sóc khách hàng để nhận diện thông tin của người gọi.
- **Phiên dịch thời gian thực:** Hỗ trợ các công cụ dịch thuật chuyển đổi giọng nói trực tiếp.
- **Ghi âm và biên bản cuộc họp:** Tự động chuyển đổi nội dung các buổi họp hoặc phỏng vấn.

vấn thành văn bản.

2.3.2 Mô hình SSD Caffe và ArcFace

Giới thiệu

Single Shot MultiBox Detector (SSD)

Là một trong những mô hình nổi bật trong lĩnh vực phát hiện đối tượng (Object Detection) với tốc độ và độ chính xác cao. SSD được giới thiệu lần đầu vào năm 2016 bởi nhóm nghiên cứu của Wei Liu, kéo theo sự thay đổi lớn trong việc phát hiện đối tượng theo thời gian thực. Khác với các mô hình truyền thống như Faster R-CNN phải trải qua hai giai đoạn (proposal + classification), SSD thực hiện tất cả trong một lần (single shot), giúp tăng tốc độ đáng kể mà không cần vùng đề xuất (Region Proposal).

SSD sử dụng kiến trúc mạng tích chập (Convolutional Neural Network - CNN) để dự đoán bounding boxes và xác định lớp của các đối tượng trực tiếp từ ảnh đầu vào, thay vì phải qua bước chọn vùng như các mô hình trước. Điều này giúp SSD có khả năng xử lý ảnh thời gian thực, là lựa chọn lý tưởng cho các ứng dụng yêu cầu tốc độ cao như camera giám sát, xe tự lái, và các thiết bị IoT.

ArcFace

ArcFace là một phương pháp tiên tiến trong nhận diện khuôn mặt, được giới thiệu bởi nhóm nghiên cứu của Microsoft Research vào năm 2019. ArcFace tối ưu hóa khoảng cách góc (Angular Distance) giữa các vector đặc trưng của khuôn mặt thay vì khoảng cách Euclidean thông thường. Phương pháp này giúp cải thiện độ phân biệt giữa các lớp, giúp mô hình nhận diện chính xác hơn ngay cả khi khuôn mặt có sự thay đổi về góc độ hoặc biểu cảm.

ArcFace được xây dựng dựa trên nền tảng mạng nơ-ron sâu (Deep Neural Networks) và ứng dụng hàm mất mát ArcMargin, giúp tối ưu hóa khoảng cách giữa các đặc trưng của khuôn mặt, tăng cường khả năng nhận diện ở khoảng cách xa và với các điều kiện ánh sáng phức tạp.

Cấu trúc và kiến trúc

Single Shot MultiBox Detector (SSD)

Kiến trúc của SSD dựa trên mô hình VGG-16 (hoặc ResNet) làm backbone để trích xuất đặc trưng. Sau phần backbone là các layers CNN bổ sung nhằm mục đích dự đoán bounding boxes ở nhiều kích thước khác nhau. SSD hoạt động trên nhiều feature maps với kích thước giảm dần, cho phép phát hiện đối tượng ở các tỉ lệ khác nhau:

1. **Backbone Network:** Thông thường là VGG-16 hoặc ResNet.
2. **Extra Feature Layers:** Các convolutional layers bổ sung để phát hiện đối tượng ở nhiều kích cỡ khác nhau.
3. **Multi-scale Feature Maps:** SSD không chỉ dựa vào feature map cuối cùng mà còn tận dụng các feature map trung gian để phát hiện đối tượng nhỏ và lớn.
4. **Default Boxes:** Tại mỗi vị trí trên feature map, SSD dự đoán nhiều bounding boxes với các tỉ lệ khác nhau.

ArcFace

- **Backbone Network:** Sử dụng các mạng CNN mạnh mẽ như ResNet50, ResNet100 để trích xuất đặc trưng.

- **ArcMargin Loss:** Hàm mất mát tối ưu hóa theo khoảng cách góc, giúp tăng cường sự phân biệt giữa các lớp khuôn mặt.
- **Feature Normalization:** Chuẩn hóa vector đặc trưng để mô hình dễ dàng tính toán khoảng cách góc.
- **Fully Connected Layers:** Kết nối với lớp đầu ra để dự đoán danh tính.

Phương thức hoạt động

Single Shot MultiBox Detector (SSD)

SSD hoạt động dựa trên ý tưởng chia ảnh thành nhiều vùng nhỏ trên các feature maps, từ đó dự đoán bounding box và phân loại trực tiếp tại mỗi vùng. SSD định nghĩa một tập hợp các 'default boxes' với nhiều tỷ lệ và tỉ lệ khung hình khác nhau tại mỗi ô của feature map:

1. **Phát hiện đa tỉ lệ:** Thông qua nhiều feature maps với các kích thước khác nhau.
2. **Phân loại và định vị:** Tại mỗi điểm trên feature map, mô hình vừa dự đoán vị trí (localization) vừa dự đoán lớp (classification).
3. **Non-Maximum Suppression (NMS):** Được sử dụng để loại bỏ các bounding boxes trùng lặp, giữ lại những khung có độ tin cậy cao nhất.

ArcFace

- **Trích xuất đặc trưng:** Sử dụng CNN để lấy các vector đặc trưng.
- **Chuẩn hóa vector:** Áp dụng chuẩn hóa để đưa vector về cùng không gian.
- **Khoảng cách góc:** Tính toán dựa trên ArcMargin để phân biệt các vector tương ứng với các khuôn mặt khác nhau.

Tính năng nổi bật

Single Shot MultiBox Detector (SSD)

- **Tốc độ cao:** Vì thực hiện tất cả trong một bước duy nhất (single shot).
- **Phát hiện đa tỉ lệ:** Khả năng phát hiện đối tượng ở nhiều kích thước khác nhau.
- **Không cần Region Proposal:** Giảm tải tính toán so với Faster R-CNN.
- **Thích hợp với thời gian thực:** Hiệu quả trên các thiết bị hạn chế tài nguyên.

ArcFace

- **Độ chính xác cao:** Nhận diện tốt hơn trong điều kiện ánh sáng phức tạp và góc nhìn thay đổi.
- **Ổn định:** Hiệu quả trong các hệ thống giám sát và nhận diện thời gian thực.
- **Thích hợp với cơ sở dữ liệu lớn:** Dễ dàng mở rộng mô hình mà không giảm hiệu năng.

Ứng dụng thực tế



Single Shot MultiBox Detector (SSD)

- Camera giám sát an ninh thời gian thực.
- Phát hiện phương tiện trong giao thông thông minh.
- Nhận diện đối tượng trong robot tự hành.
- Theo dõi đối tượng trong video streaming.

ArcFace

- Hệ thống chấm công và kiểm soát ra vào.
- Giám sát an ninh thông minh.
- Nhận diện khuôn mặt trong thanh toán điện tử.
- Xác thực danh tính trong các ứng dụng di động.

3 Phân tích

3.1 Hệ thống phần cứng

3.1.1 Giám sát môi trường

Hệ thống yêu cầu khả năng thu thập dữ liệu từ môi trường, các dữ liệu này sẽ được thu thập và xử lý sử dụng các loại cảm biến khác nhau. Trong hệ thống này, nhiệt độ và độ ẩm sẽ được thu thập dựa trên *Module DHT20* và ánh sáng sẽ được thu thập dựa trên *Module cảm biến ánh sáng* (quang trở). Hệ thống cũng yêu cầu hiển thị dữ liệu, *Module LCD16x2* sẽ hỗ trợ việc hiển thị dữ liệu trực tiếp sau khi đo đạc.

Hệ thống yêu cầu chức năng bật tắt các thiết bị thông minh dựa trên dữ liệu từ môi trường, các thiết bị được lựa chọn để mô phỏng khả năng bật tắt này dựa trên việc đo đạc các thông số từ môi trường là *Module Quạt Mini* và *Module 4 đèn LED RGB*.

Hệ thống cũng yêu cầu cửa mật mã để đảm bảo an ninh ra vào. *Remote* và *Module mắt IR* sẽ được áp dụng vào hệ thống cửa khóa mật mã, động cơ *Servo SG90* sẽ đóng vai trò là chốt khóa cửa của hệ thống.

Board mạch điều khiển chính của hệ thống sẽ là Yolo: Bit sử dụng Vi điều khiển *ESP32 Wrover B* đóng vai trò là đầu não của hệ thống phần cứng, đây là trung tâm nhận, xử lý, điều khiển và kết nối với Server Adafruit, từ đó thực hiện việc giao tiếp với Database.

3.2 Phần mềm

3.2.1 Yêu cầu người dùng

3.2.1.1 Yêu cầu chức năng

Giám sát môi trường

- Có khả năng thu thập dữ liệu môi trường.
- Có khả năng xử lý dữ liệu như tính toán chỉ số, xu hướng môi trường và hiển thị dữ liệu.
- Hệ thống có thể kích hoạt các thiết bị thông minh dựa trên dữ liệu môi trường.
- Gửi cảnh báo nếu phát hiện môi trường bất thường.

Cửa mật mã

- Có khả năng mở cửa bằng mật mã.
- Có giới hạn số lần nhập sai.
- Có thể thay đổi mật mã.
- Có thể ghi lại lịch sử truy cập.
- Báo động khi có xâm nhập trái phép.

Điều khiển thiết bị bằng giọng nói

- Nhận diện, xử lý lệnh giọng nói và phản hồi khi nhận lệnh.
- Lưu trữ lịch sử điều khiển.
- Báo lỗi khi không nhận diện được giọng nói.

Nhận dạng khuôn mặt FaceAI

- Huấn luyện AI để nhận dạng khuôn mặt mới.
- Có thể nhận dạng khuôn mặt và xác thực người dùng.
- Gửi cảnh báo khi phát hiện khuôn mặt lạ.
- Ghi lại nhật ký truy cập.

3.2.1.2 Yêu cầu phi chức năng

Giám sát môi trường

- Độ chính xác cao.



- Tốc độ cập nhật nhanh.

Cửa mật mã

- Bảo mật.

Điều khiển thiết bị bằng giọng nói

- Độ chính xác cao.
- Bảo mật.
- Độ trễ thấp.

Nhận dạng khuôn mặt FaceAI

- Độ chính xác cao.
- Bảo mật.
- Hiệu suất.

3.2.2 Use Case Detail

3.2.2.1 Giám sát môi trường

Use Case ID:	1
Use Case Name:	Thu thập dữ liệu môi trường
Create By:	Đồng Mạnh Cường
Last updated by:	Đồng Mạnh Cường
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng, hệ thống
Description:	Hệ thống thu thập dữ liệu từ các cảm biến đo nhiệt độ, độ ẩm, chất lượng không khí, mức độ ô nhiễm nước,... và lưu vào cơ sở dữ liệu.
Preconditions:	Cảm biến môi trường đã được lắp đặt và kết nối hệ thống.
Postconditions:	Dữ liệu môi trường được lưu trữ thành công.
Normal Flow:	1. Người dùng đăng nhập vào hệ thống giám sát môi trường và vào giao diện Thu thập dữ liệu. 2. Người dùng chọn loại dữ liệu cần thu thập (ví dụ: nhiệt độ, độ ẩm, v.v.). 3. Người dùng cấu hình thời gian thu thập định kỳ. 4. Người dùng nhấn Lưu cài đặt. 5. Hệ thống bắt đầu thu thập dữ liệu từ cảm biến theo cài đặt của người dùng. 6. Hệ thống kiểm tra tính hợp lệ của dữ liệu thu thập. 7. Nếu dữ liệu hợp lệ, hệ thống lưu vào cơ sở dữ liệu. 8. Hệ thống hiển thị dữ liệu thu thập được trên giao diện.
Alternative Flow:	Bước 3: Nếu người dùng không chọn chế độ thu thập, hệ thống sẽ mặc định sử dụng lịch trình cài đặt sẵn.
Exceptions Flow:	Bước 6: Nếu hệ thống không nhận được dữ liệu từ cảm biến (do lỗi kết nối hoặc cảm biến hỏng), hệ thống hiển thị cảnh báo cho người dùng; Bước 7: Nếu dữ liệu không hợp lệ (ví dụ: giá trị ngoài phạm vi hợp lý), hệ thống loại bỏ dữ liệu và ghi log lỗi.



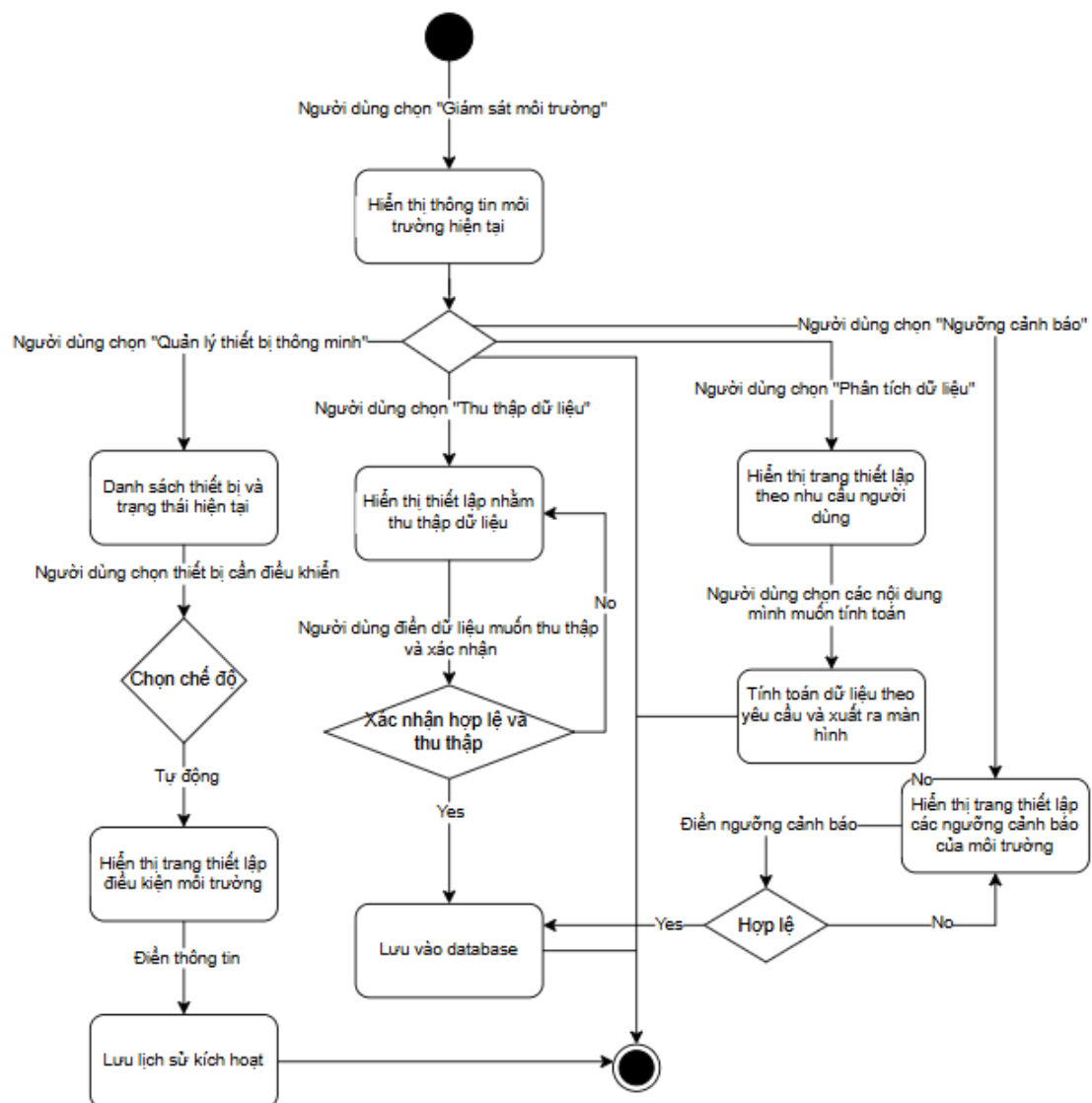
Use Case ID:	2
Use Case Name:	Xử lý và hiển thị dữ liệu môi trường
Create By:	Đồng Mạnh Cường
Last updated by:	Đồng Mạnh Cường
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng, Hệ thống
Description:	Hệ thống xử lý dữ liệu thu thập, tính toán các chỉ số môi trường, xu hướng ô nhiễm, và hiển thị trên giao diện để người dùng theo dõi.
Preconditions:	Hệ thống đã thu thập đủ dữ liệu môi trường.
Postconditions:	Dữ liệu môi trường được xử lý và hiển thị dưới dạng bảng, biểu đồ hoặc báo cáo.
Normal Flow:	- Người dùng đăng nhập vào hệ thống giám sát môi trường và vào giao diện Phân tích dữ liệu. - Người dùng chọn loại dữ liệu muốn xem (ví dụ: nhiệt độ, độ ẩm, v.v.). - Người dùng chọn khoảng thời gian cần xem dữ liệu (hôm nay, tuần này, tháng này, hoặc tùy chỉnh). - Hệ thống lấy dữ liệu từ cơ sở dữ liệu và thực hiện các phân tích sau: - Tính toán các chỉ số yêu cầu. - Xác định xu hướng môi trường (ví dụ: mức độ ô nhiễm tăng hay giảm theo thời gian). - Hệ thống hiển thị dữ liệu dưới dạng: - Bảng thống kê (giá trị trung bình, cao nhất, thấp nhất). - Biểu đồ trực quan (đường, cột, nhiệt độ, bản đồ nhiệt). - Báo cáo nhanh (đưa ra đánh giá tổng quan về môi trường trong thời gian đã chọn). - Người dùng có thể lọc và so sánh dữ liệu theo nhu cầu.
Alternative Flow:	Bước 2: Nếu người dùng không chọn loại dữ liệu, hệ thống sẽ hiển thị tổng quan tất cả các chỉ số.
Exceptions Flow:	Bước 4: Nếu không có dữ liệu trong khoảng thời gian được chọn, hệ thống hiển thị thông báo “Không có dữ liệu khả dụng”. Bước 5: Nếu hệ thống gặp lỗi khi phân tích dữ liệu, hệ thống hiển thị thông báo lỗi và đề xuất thử lại sau.



Use Case ID:	3
Use Case Name:	Kích hoạt thiết bị thông minh dựa trên dữ liệu môi trường
Create By:	Đồng Mạnh Cường
Last updated by:	Đồng Mạnh Cường
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng, Hệ thống
Description:	Hệ thống có thể tự động kích hoạt các thiết bị thông minh dựa trên ngưỡng giá trị môi trường được cấu hình trước.
Preconditions:	1. Hệ thống đã có dữ liệu môi trường để phân tích. 2. Các thiết bị thông minh đã được kết nối với hệ thống.
Postconditions:	1. Thiết bị thông minh được kích hoạt hoặc tắt theo điều kiện môi trường. 2. Hệ thống ghi nhận lịch sử hoạt động của thiết bị.
Normal Flow:	1. Người dùng đăng nhập vào hệ thống giám sát môi trường và vào giao diện Quản lý thiết bị thông minh. 2. Người dùng chọn thiết bị cần cấu hình (ví dụ: máy lọc không khí, hệ thống tưới nước). 3. Người dùng thiết lập ngưỡng kích hoạt, ví dụ: • Nếu độ ẩm < 30% → Bật hệ thống phun sương. • Nếu nhiệt độ > 35°C → Bật quạt thông gió. 4. Người dùng nhấn Lưu cài đặt. 5. Hệ thống lưu lại cài đặt và bắt đầu giám sát dữ liệu môi trường. 6. Khi dữ liệu môi trường đạt ngưỡng kích hoạt, hệ thống tự động bật thiết bị. 7. Khi điều kiện môi trường trở lại mức an toàn, hệ thống tự động tắt thiết bị. 8. Hệ thống ghi lại lịch sử kích hoạt thiết bị.
Alternative Flow:	Bước 2: a. Người dùng vào giao diện Điều khiển thiết bị. b. Người dùng chọn thiết bị và nhấn Bật/Tắt theo nhu cầu. Bước 3: Nếu người dùng không thiết lập ngưỡng kích hoạt, hệ thống sử dụng giá trị mặc định.
Exceptions Flow:	Bước 5: Nếu hệ thống không thể lưu cài đặt, hiển thị thông báo lỗi và yêu cầu thử lại. Bước 6: Nếu thiết bị không phản hồi hoặc mất kết nối, hệ thống thông báo lỗi cho người dùng. Bước 8: Nếu hệ thống không ghi lại lịch sử hoạt động, hiển thị thông báo lỗi và yêu cầu thử lại.



Use Case ID:	4
Use Case Name:	Gửi cảnh báo khi phát hiện môi trường bất thường
Create By:	Đồng Mạnh Cường
Last updated by:	Đồng Mạnh Cường
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng, Hệ thống
Description:	Hệ thống giám sát dữ liệu môi trường theo thời gian thực và tự động gửi cảnh báo khi phát hiện giá trị vượt ngưỡng quy định.
Preconditions:	1. Hệ thống đang thu thập dữ liệu môi trường theo thời gian thực. 2. Ngưỡng cảnh báo đã được cấu hình trong hệ thống.
Postconditions:	Cảnh báo được gửi đến người dùng qua hệ thống (email, SMS, ứng dụng di động, loa cảnh báo, v.v.).
Normal Flow:	1. Người dùng chọn Ngưỡng cảnh báo để setup ngưỡng báo hiệu. 2. Hệ thống phân tích dữ liệu môi trường theo thời gian thực. 3. Khi một chỉ số môi trường vượt ngưỡng an toàn (ví dụ: CO ₂ tăng đột biến, nhiệt độ quá cao), hệ thống kích hoạt cảnh báo. 4. Hệ thống gửi thông báo đến người dùng. 5. Hệ thống ghi nhận lịch sử cảnh báo.
Alternative Flow:	Không có.
Exceptions Flow:	Bước 4: Nếu hệ thống không thể gửi cảnh báo, nó sẽ ghi log lỗi và thử lại sau.



Hình 1: Activity diagram của Giám sát môi trường



3.2.2.2 Cửa mật mã

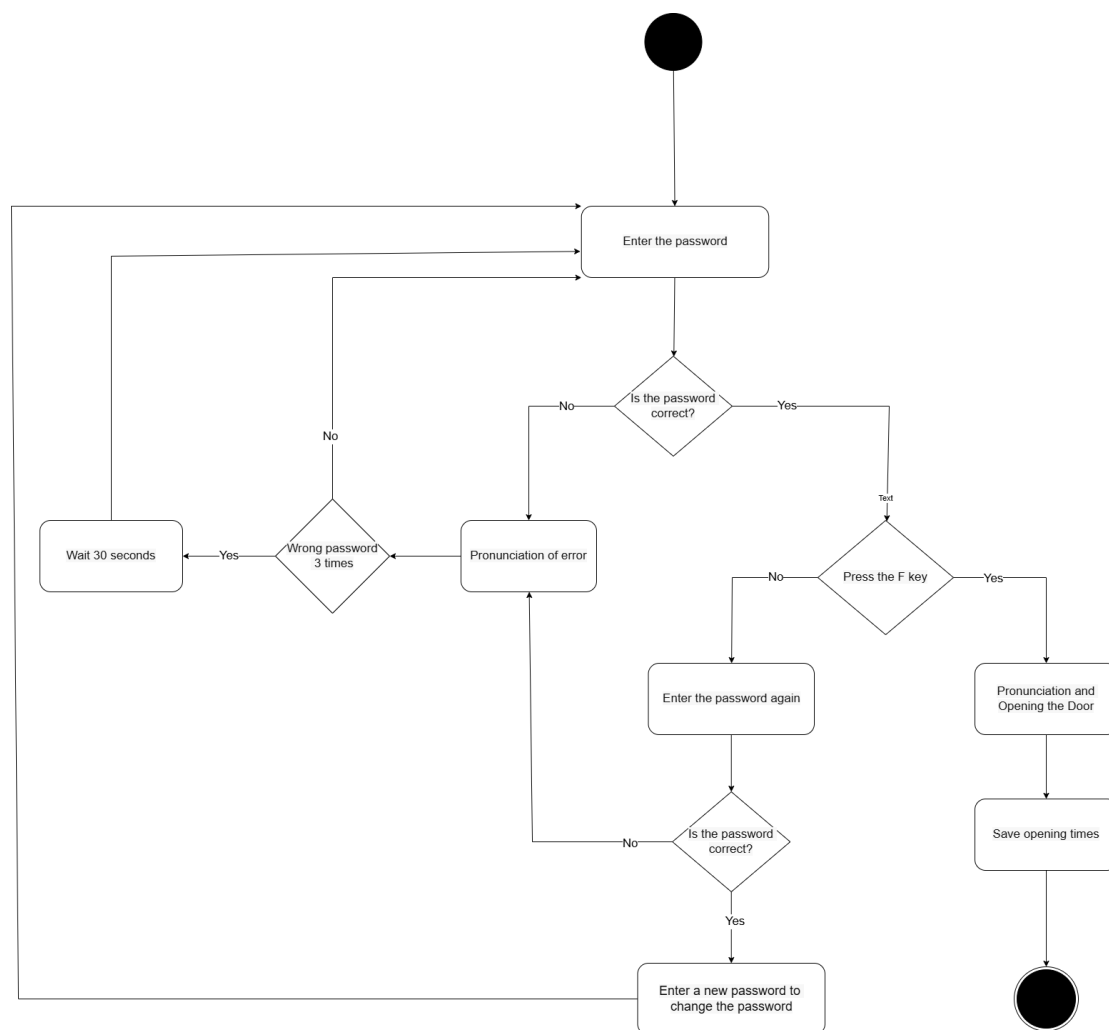
Use Case ID:	5
Use Case Name:	Nhập mật khẩu và kiểm tra
Create By:	Đậu Ngọc Quân
Last updated by:	Đậu Ngọc Quân
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng
Description:	Hệ thống cho phép người dùng nhập mật khẩu để kiểm tra và mở cửa. Nếu mật khẩu không đúng, hệ thống sẽ phát âm báo lỗi.
Preconditions:	Hệ thống ở trạng thái chờ nhập mật khẩu.
Postconditions:	Nếu nhập đúng, cửa mở trong vài giây rồi đóng lại.
Normal Flow:	1. Người dùng nhập mật khẩu bằng remote. 2. Nhấn nút F để xác nhận. 3. Hệ thống kiểm tra mật khẩu. 4. Nếu đúng, mở cửa. 5. Nếu sai, phát âm báo lỗi.
Alternative Flow:	Không có.
Exceptions Flow:	Bước 5: Nếu nhập sai 3 lần, hệ thống khóa tạm thời 30 giây

Use Case ID:	6
Use Case Name:	Mở cửa và Đóng cửa
Create By:	Đậu Ngọc Quân
Last updated by:	Đậu Ngọc Quân
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng, Hệ thống
Description:	Mở cửa và Đóng cửa sau khi nhập mật khẩu.
Preconditions:	Mật khẩu đã nhập đúng.
Postconditions:	Cửa đóng hoàn toàn, hệ thống trở về trạng thái chờ.
Normal Flow:	1. Hệ thống điều khiển servo để mở cửa. 2. Cửa mở trong vài giây được setup trước. 3. Sau vài giây, cửa tự động đóng.
Alternative Flow:	Không có.
Exceptions Flow:	Nếu có lỗi phần cứng, hệ thống hiển thị cảnh báo.



Use Case ID:	7
Use Case Name:	Thay đổi mật khẩu
Create By:	Đậu Ngọc Quân
Last updated by:	Đậu Ngọc Quân
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng
Description:	Thay đổi mật khẩu.
Preconditions:	Đã nhập đúng mật khẩu xác nhận.
Postconditions:	Mật khẩu mới được lưu, hệ thống trở về trạng thái chờ.
Normal Flow:	1. Người dùng nhập mật khẩu hiện tại 2 lần liên tiếp. 2. Nhập mật khẩu mới. 3. Xác nhận. 4. Hệ thống lưu mật khẩu mới.
Alternative Flow:	Không có.
Exceptions Flow:	Nếu mật khẩu mới không hợp lệ, hiển thị thông báo lỗi.

Use Case ID:	8
Use Case Name:	Lưu lịch sử
Create By:	Đậu Ngọc Quân
Last updated by:	Đậu Ngọc Quân
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Hệ thống
Description:	Lưu lịch sử mỗi hoạt động truy cập cửa.
Preconditions:	Có hoạt động truy cập đến cửa.
Postconditions:	Hoạt động được lưu lại.
Normal Flow:	1. Người dùng tương tác với cửa. 2. Hệ thống ghi nhận lại mọi hoạt động.
Alternative Flow:	Không có.
Exceptions Flow:	Không có.



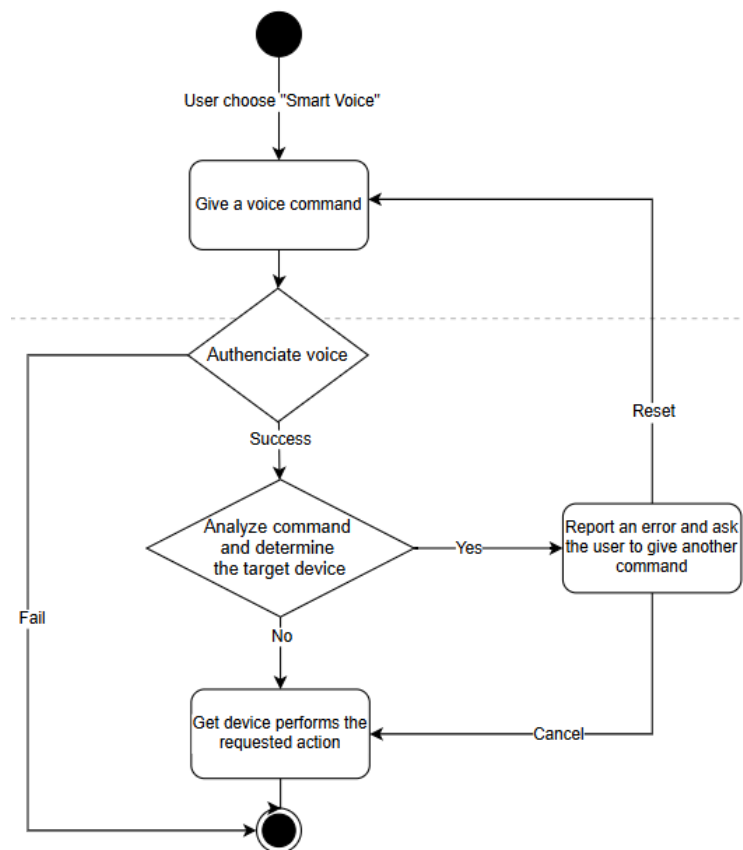
Hình 2: Activity diagram của Cửa mật mã

3.2.2.3 Điều khiển thiết bị bằng giọng nói

Use Case ID:	9
Use Case Name:	Điều khiển thiết bị.
Create By:	Đỗ Tuấn Đạt
Last updated by:	Đỗ Tuấn Đạt
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng
Description:	Người dùng có thể điều khiển các thiết bị trong nhà (hệ thống chiếu sáng, hệ thống điều hòa) bằng giọng nói.
Preconditions:	1. Hệ thống của các thiết bị trong nhà đã được kết nối với hệ thống trung tâm. 2. Hệ thống nhận diện giọng nói đã được kích hoạt và sẵn sàng nhận lệnh. 3. Cảm biến nhiệt độ và ánh sáng có thể hoạt động bình thường.
Postconditions:	Điều khiển thiết bị bằng giọng nói thành công.
Normal Flow:	1. Người dùng ra lệnh bằng giọng nói. 2. Hệ thống nhận diện giọng nói và phân tích câu lệnh. 3. Hệ thống gửi yêu cầu đến bộ điều khiển trung tâm. 4. Bộ điều khiển trung tâm gửi tín hiệu đến hệ thống của thiết bị. 5. Các thiết bị sẽ được thay đổi tùy theo câu lệnh.
Alternative Flow:	Bước 1: Hệ thống yêu cầu người dùng lặp lại lệnh nếu hệ thống không nhận diện được giọng nói.
Exceptions Flow:	Bước 2: Nếu thông số yêu cầu thiết bị nằm ngoài phạm vi cho phép (quá nóng, quá lạnh, quá sáng) thì hệ thống sẽ báo lỗi và thông báo cho người dùng đặt lại lệnh

Use Case ID:	10
Use Case Name:	Lưu trữ lịch sử điều khiển
Create By:	Đỗ Tuấn Đạt
Last updated by:	Đỗ Tuấn Đạt
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Hệ thống
Description:	Hệ thống sẽ lưu trữ lại lịch sử điều khiển các thiết bị.
Preconditions:	Cơ sở dữ liệu có sẵn và dung lượng cho bộ nhớ vẫn còn.
Postconditions:	Câu lệnh tương tác sẽ được lưu trữ lại trên cơ sở dữ liệu.
Normal Flow:	1. Người dùng ra lệnh bằng giọng nói. 2. Nếu hệ thống nhận diện và phân tích thành công, câu lệnh sẽ được lưu trữ lại.
Alternative Flow:	Không có.
Exceptions Flow:	Nếu câu lệnh không thành công, hệ thống sẽ báo lỗi và không ghi nhận lại câu lệnh.

Use Case ID:	11
Use Case Name:	Báo lỗi khi không nhận diện được giọng nói
Create By:	Đỗ Tuấn Đạt
Last updated by:	Đỗ Tuấn Đạt
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Hệ thống.
Description:	Khi hệ thống không thể nhận diện được giọng nói, nó sẽ báo lỗi và yêu cầu người dùng thực hiện lại.
Preconditions:	Hệ thống nhận diện giọng nói đã được kích hoạt và sẵn sàng hoạt động.
Postconditions:	Hệ thống báo lỗi cho người dùng và sẵn sàng để nhận lệnh tiếp theo.
Normal Flow:	1. Người dùng ra lệnh bằng giọng nói. 2. Hệ thống sẽ nhận diện giọng nói và phân tích lệnh. 3. Trong trường hợp này, hệ thống sẽ không thể nhận ra câu lệnh. 4. Hệ thống báo lại lỗi cho người dùng để thực hiện lại hành động.
Alternative Flow:	Nếu người dùng thực hiện lại câu lệnh, hệ thống sẽ bắt đầu lại việc phân tích.
Exceptions Flow:	Không có.



Hình 3: Activity diagram của Điều khiển thiết bị bằng giọng nói

3.2.2.4 Nhận diện khuôn mặt - FaceAI

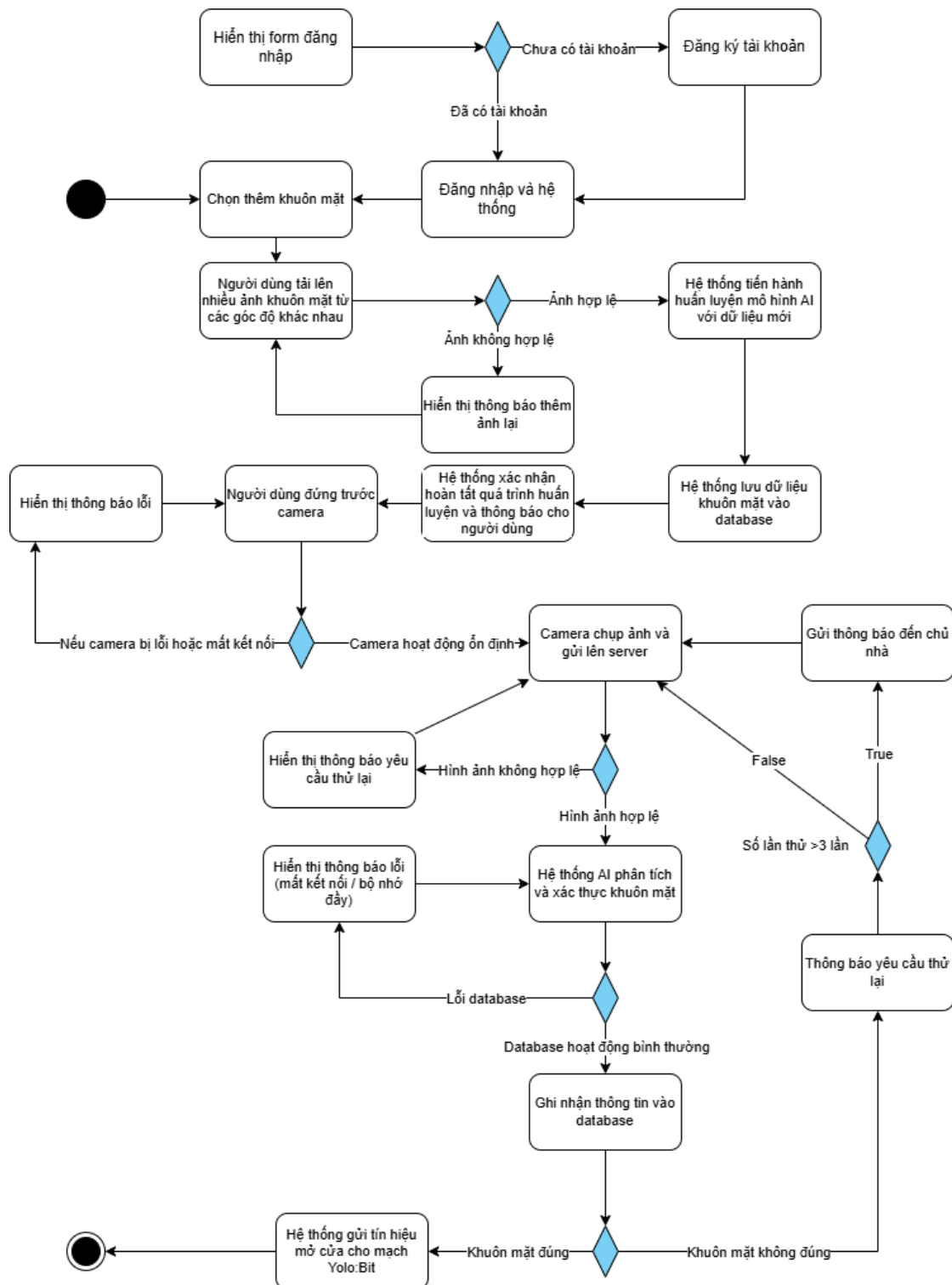
Use Case ID:	12
Use Case Name:	Huấn luyện AI để nhận diện khuôn mặt mới.
Create By:	Hồ Quang Long
Last updated by:	Hồ Quang Long
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng
Description:	Người dùng cung cấp dữ liệu hình ảnh để huấn luyện hệ thống có thể nhận diện khuôn mặt của họ.
Preconditions:	Người dùng có tài khoản trong hệ thống.
Postconditions:	Huấn luyện AI thành công và lưu trữ vào database.
Normal Flow:	1. Người dùng cung cấp nhiều ảnh khuôn mặt của mình từ các góc độ khác nhau. 2. Hệ thống huấn luyện mô hình với dữ liệu mới. 3. Hệ thống lưu thông tin vào database và xác nhận hoàn tất.
Alternative Flow:	Không có.
Exceptions Flow:	Bước 1: Ảnh tải lên không hợp lệ hoặc chất lượng thấp sẽ báo lỗi

Use Case ID:	13
Use Case Name:	Xác thực danh tính
Create By:	Hồ Quang Long
Last updated by:	Hồ Quang Long
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng
Description:	Hệ thống sử dụng AI để nhận diện khuôn mặt, nếu đúng người đã đăng ký dữ liệu thì cửa sẽ tự động mở hoặc thực hiện một số chức năng yêu cầu xác thực khuôn mặt.
Preconditions:	1. Hệ thống đã có dữ liệu và đã được huấn luyện để nhận diện khuôn mặt của người dùng. 2. Camera đang hoạt động và kết nối với server.
Postconditions:	Xác thực khuôn mặt thành công.
Normal Flow:	1. Người dùng đứng trước camera. 2. Camera chụp ảnh và gửi lên server để AI xử lý. 3. Nếu khuôn mặt được nhận diện đúng, hệ thống gửi tín hiệu mở cửa cho mạch Yolo:Bit. 4. Cửa mở và hệ thống hiển thị thông báo thành công.
Alternative Flow:	Nếu nhận diện thất bại, hệ thống sẽ yêu cầu thử lại.
Exceptions Flow:	Camera bị lỗi hoặc mất kết nối. Khuôn mặt không được nhận diện do ánh sáng yếu hoặc góc chụp sai.



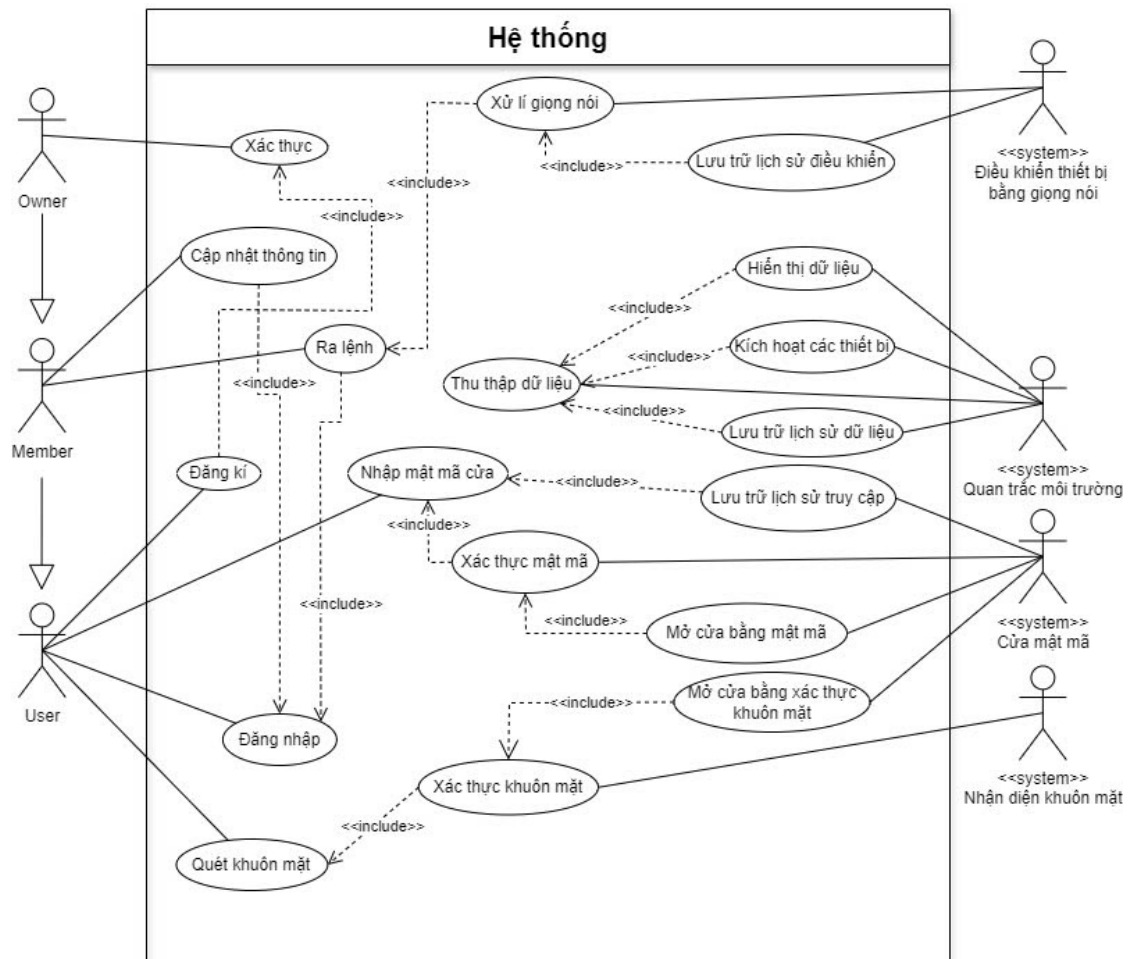
Use Case ID:	14
Use Case Name:	Gửi cảnh báo khi phát hiện khuôn mặt lạ
Create By:	Hồ Quang Long
Last updated by:	Hồ Quang Long
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng
Description:	Nếu hệ thống phát hiện khuôn mặt không có trong danh sách đã đăng ký, nó sẽ gửi cảnh báo đến chủ nhà.
Preconditions:	Hệ thống đã có danh sách khuôn mặt đã đăng ký.
Postconditions:	Chủ nhà nhận được cảnh báo.
Normal Flow:	1. Hệ thống quét và phát hiện khuôn mặt. 2. Nếu khuôn mặt không khớp với danh sách đã đăng ký, hệ thống ghi nhận hình ảnh và gửi thông báo đến chủ nhà. 3. Chủ nhà có thể kiểm tra hình ảnh và quyết định có cho phép truy cập hay không.
Alternative Flow:	Nếu nhận diện sai, chủ nhà có thể cập nhật danh sách khuôn mặt hợp lệ.
Exceptions Flow:	Hệ thống mất kết nối internet, không thể gửi cảnh báo. Camera bị lỗi, không nhận diện được khuôn mặt

Use Case ID:	15
Use Case Name:	Ghi lại nhật ký truy cập
Create By:	Hồ Quang Long
Last updated by:	Hồ Quang Long
Date Created:	18/2/2025
Date last updated:	20/2/2025
Actors:	Người dùng
Description:	Hệ thống lưu trữ thông tin về các lần nhận diện khuôn mặt, bao gồm thời gian, trạng thái nhận diện, và hình ảnh.
Preconditions:	1. Hệ thống đã thiết lập nhận diện khuôn mặt. 2. Database có đủ dung lượng để lưu trữ dữ liệu. 3. Camera và server hoạt động bình thường.
Postconditions:	Hệ thống lưu lại lịch sử truy cập thành công
Normal Flow:	1. Người dùng đứng trước camera. 2. Hệ thống nhận diện khuôn mặt và xác minh danh tính. 3. Nếu nhận diện thành công, thông tin sẽ được lưu vào database.
Alternative Flow:	Nếu hệ thống không thể xác định danh tính, thông tin vẫn được lưu lại nhưng đánh dấu là "người lạ".
Exceptions Flow:	Hệ thống mất kết nối với database, không thể lưu dữ liệu. Bộ nhớ đầy, không thể ghi thêm nhật ký. Camera bị lỗi, không thu được hình ảnh.



Hình 4: Activity diagram của Điều khiển thiết bị bằng giọng nói

3.2.3 Use-case diagram



Hình 5: Use case diagram

3.3 Phần trí tuệ nhân tạo

3.3.1 Nhận diện giọng nói

1. Bài toán & Yêu cầu:

Chuyển giọng nói tiếng Việt thành văn bản để điều khiển thiết bị.

- Đầu vào: tín hiệu âm thanh mono từ microphone, sampling rate (ví dụ: 16 kHz hoặc theo mặc định của thư viện), khung âm thanh tối đa 7 giây (theo `phrase_time_limit=7` trong mã nguồn).
- Đầu ra: JSON chứa chuỗi `transcript` (kết quả nhận diện) hoặc trường `error` (thông báo lỗi), được xử lý bởi backend Node.js.

2. Tiêu chí chất lượng:

- **Word Error Rate (WER)** $\leq 25\%$ trên tập dữ liệu thử nghiệm tiếng Việt với nhiều kiểu giọng nói.

3. Lựa chọn mô hình & Xác thực thực tế:

- **Thư viện thu âm & hiệu chuẩn:** `speech_recognition` của Python. Sử dụng `r.adjust_for_ambient_noise(source, duration=1)` để hiệu chỉnh nhiễu nền trong 1 giây.
- **Engine nhận diện:** Google Web Speech API thông qua `r.recognize_google(audio, language='vi-VN')`.
- **Cấu hình timeout:** `r.listen(source, timeout=3, phrase_time_limit=7)`. Thời gian chờ bắt đầu nói là 3 giây, thời gian ghi âm tối đa cho một cụm từ là 7 giây.

VIVOSSPK03_R177.wav	NHƯNG HỒNG LẠI CĂNG NGẮC NHIÊN HƠN NỮA	NHƯ HỒNG LẠI CĂNG NGẮC NHIÊN HƠN NỮA	✗
VIVOSSPK03_R178.wav	CHỊ HẰNG CHO CON SÁO ẦM MỘT CHẾ ĐỘ ĐẶC BIỆT	CHỊ HẰNG CHO CON SÁO ẦM MỘT CHẾ ĐỘ ĐẶC BIỆT	✓
VIVOSSPK03_R179.wav	MỘNG TUYẾT ĐỒNG HỒ	MỘNG TUYẾT ĐỒNG HỒ	✓
VIVOSSPK03_R181.wav	NHẦM MẮT CẠM CHỊU	NHẦM MẮT CẠM CHỊU	✓
VIVOSSPK03_R182.wav	MƯA NHÀ TRONG DỰ ẦM	MƯA NHÀ TRONG DỰ ẦM	✓
VIVOSSPK03_R183.wav	CÁC NHÀ NGHIÊN CỨU HI VỌNG	CÁC NHÀ NGHIÊN CỨU HI VỌNG	✗
VIVOSSPK03_R188.wav	VỤ HÒA HOÀN NÀY TUY KHÔNG GÂY THIẾT HẠI LỚN	VỤ HÒA HOÀN NÀY TUY KHÔNG GÂY THIẾT HẠI LỚN	✓
VIVOSSPK03_R193.wav	CỦA TỈNH NÔNG MỒI VÀ SỰ TỎ MỒ	CỦA TỈNH NÔNG MỒI VÀ SỰ TỎ MỒ	✓
VIVOSSPK03_R194.wav	SẼ NHẢY RA NHỮNG DẤU HIỆU BÁO TRƯỚC TRÊN ĐƯỜNG	SẼ NHẢY RA NHỮNG DẤU HIỆU BÁO TRƯỚC TRÊN ĐƯỜNG	✓
VIVOSSPK03_R195.wav	NHỮNG CON VẬT NÀY ĐÃ BỊ GIẾT VÀ ĐỐT	NHỮNG CON VẬT NÀY ĐÃ BỊ GIẾT VÀ ĐỐT	✓
VIVOSSPK03_R199.wav	VÍ DỤ NGƯỜI TA KHÓ TÌM RA SỰ KHÁC BIỆT	VÍ DỤ NGƯỜI TA KHÓ TÌM RA SỰ KHÁC BIỆT	✓
VIVOSSPK03_R200.wav	VÀ NHỮNG PHIM NÀY ĐÃ TRỞ NÊN BẤT HỮ	VÀ NHỮNG PHIM NÀY ĐÃ TRỞ NÊN BẤT HỮ	✓
Tổng số file đã xử lý: 420			
Số câu nhận diện đúng: 334/420 → Độ chính xác: 79.52%			
--- thống kê thời gian trễ (cho 420 lượt gọi API thành công hoặc không hiệu) ---			
Thời gian trễ trung bình: 0.852 giây			
Thời gian trễ nhỏ nhất: 0.325 giây			
Thời gian trễ lớn nhất: 3.431 giây			
Hoàn thành.			

Hình 6: Minh họa kết quả đánh giá nhận diện giọng nói trên tập dữ liệu thử nghiệm.

4. Kết quả thử nghiệm

- Sử dụng một tập dữ liệu gồm 420 câu lệnh/phát biểu tiếng Việt (đa dạng giọng nói, có thể bao gồm các trường hợp ngắt quãng, nhiễu nhẹ để đánh giá:
 - Số câu nhận diện đúng: 334/420 câu.
 - **Độ chính xác (Accuracy):** $(334/420) \times 100\% \approx 79.52\%$.

- **Word Error Rate (WER)** trung bình: $100\% - 79.52\% \approx 20.48\%$. Kết quả này đạt yêu cầu đề ra ($WER \leq 25\%$).

- **Latency (Thời gian trễ):**

- **Phần xử lý của hệ thống thời gian thực (mic_stream.py):**

- * Hiệu chuẩn môi trường: 1 s (thời gian cố định cho `adjust_for_ambient_noise`).
- * Thời gian ghi âm bởi `mic_stream.py`: Phụ thuộc vào cấu hình `phrase_time_limit=7s` và thời điểm người dùng bắt đầu nói.

- **Phần nhận diện giọng nói (Google Speech API - đánh giá trên tập VIVOS):** Sử dụng kịch bản đánh giá trên 420 tệp âm thanh của tập dữ liệu VIVOS (với script đã cập nhật để đo thời gian), thời gian trễ cho riêng các lượt gọi API nhận diện của Google (bao gồm các lượt gọi thành công hoặc không hiểu được giọng nói) được ghi nhận như sau:

- * Số lượt đo: 420.
- * Thời gian trễ trung bình: 0.852 s.
- * Thời gian trễ nhỏ nhất: 0.325 s.
- * Thời gian trễ lớn nhất: 3.431 s.

- **Độ ổn định:** Hệ thống sử dụng Google Speech API, vốn được huấn luyện trên lượng dữ liệu lớn, cho độ ổn định tương đối tốt với các giọng nói và điều kiện môi trường khác nhau. Việc hiệu chuẩn nhiều giúp cải thiện hiệu năng trong môi trường có tạp âm.

5. Kết luận:

Mô hình Google Speech API (triển khai qua `speech_recognition` Python) đạt kết quả $WER \approx 20.48\%$ trên 420 câu thử nghiệm, với thời gian trễ trung bình cho API nhận diện là 0.852 s (nhỏ nhất 0.325 s, lớn nhất 3.431 s). Các kết quả này đáp ứng yêu cầu về độ chính xác và hiệu năng cho ứng dụng tương tác giọng nói tiếng Việt thời gian thực. Backend Node.js tích hợp xử lý lệnh bằng `string-similarity` để thực thi các hành động tương ứng. Để cải thiện hơn nữa, có thể xem xét:

- Tăng cường tập lệnh mẫu trong cơ sở dữ liệu để cải thiện độ chính xác của việc khớp lệnh.
- Nghiên cứu các kỹ thuật xử lý hậu kỳ (post-processing) phức tạp hơn nếu cần thiết.
- Đánh giá các engine nhận diện giọng nói offline (ví dụ: Vosk) để giảm phụ thuộc vào kết nối internet và có thể cải thiện latency trong một số trường hợp.

3.3.2 Nhận diện khuôn mặt

1. Bài toán & Yêu cầu Nhận diện khuôn mặt qua camera cửa để mở khóa tự động.

- *Dầu vào:* frame RGB từ camera (resize 300×300 để detect).

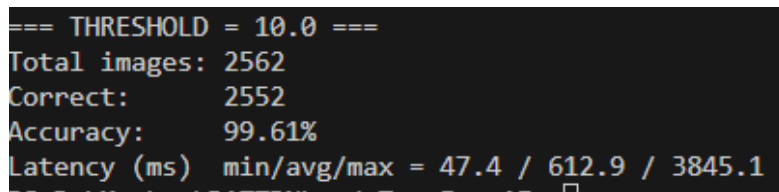
- *Đầu ra*: userID hoặc Unknown kèm khoảng cách Euclid.

2. Tiêu chí chất lượng

- **Accuracy** $\geq 90\%$ trên tập test.
- **FAR** (False Acceptance Rate) rất thấp, ưu tiên an toàn.
- **Latency** ≤ 500 ms/frame (mục tiêu ≤ 200 ms).

3. Lựa chọn mô hình & Xác thực thực tế

- **Face Detector**: DNN OpenCV với `deploy.prototxt + res10_300x300_ssd_iter_140000_fp16.caffemodel` (confidence threshold ≥ 0.6).
- **Feature Extractor**: DeepFace ArcFace $\rightarrow$ embedding 512-dim.
- **So khớp**: Euclidean distance với ngưỡng THRESHOLD (thử: 10.0).
- **Frame skipping**: SKIP_FRAMES = 10 để giảm tải CPU.



```
=== THRESHOLD = 10.0 ===
Total images: 2562
Correct:      2552
Accuracy:     99.61%
Latency (ms)  min/avg/max = 47.4 / 612.9 / 3845.1
```

Hình 7: kết quả nhận diện khuôn mặt trên tập test.

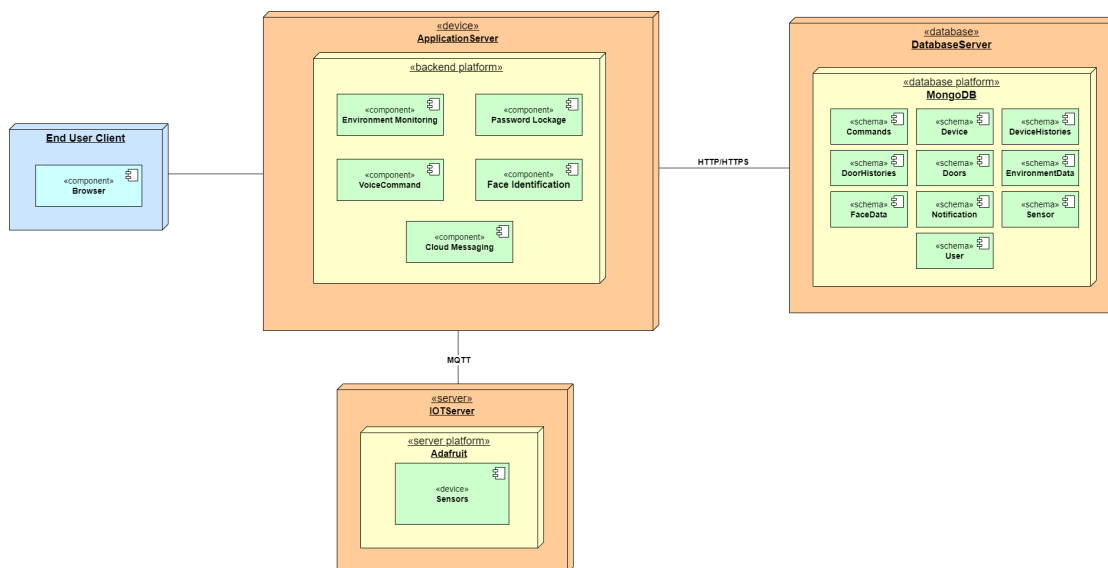
4. Kết quả thử nghiệm

- Accuracy đạt 99.61%, vượt xa ngưỡng yêu cầu ($\geq 95\%$).
- Latency trung bình ≈ 613 ms/frame, tối thiểu ≈ 47 ms, tối đa ≈ 3845 ms.

5. **Kết luận** Với độ chính xác cực cao và latency trung bình có thể chấp nhận được, ArcFace là lựa chọn phù hợp để tích hợp vào sản phẩm mở cửa tự động.

4 Thiết kế

4.1 Deployment View



Hình 8: Deployment View

Mô hình hệ thống sẽ bắt đầu tại End User Client. Khi Client sử dụng một trình duyệt để truy cập vào trang web của hệ thống, hệ thống sẽ trả về giao diện chính. Từ giao diện chính đó, người dùng có thể lựa chọn một số các tính năng đặc biệt có trong hệ thống, chẳng hạn như: Environment Monitoring, Voice Command, Password Lockage hoặc Face Identification.

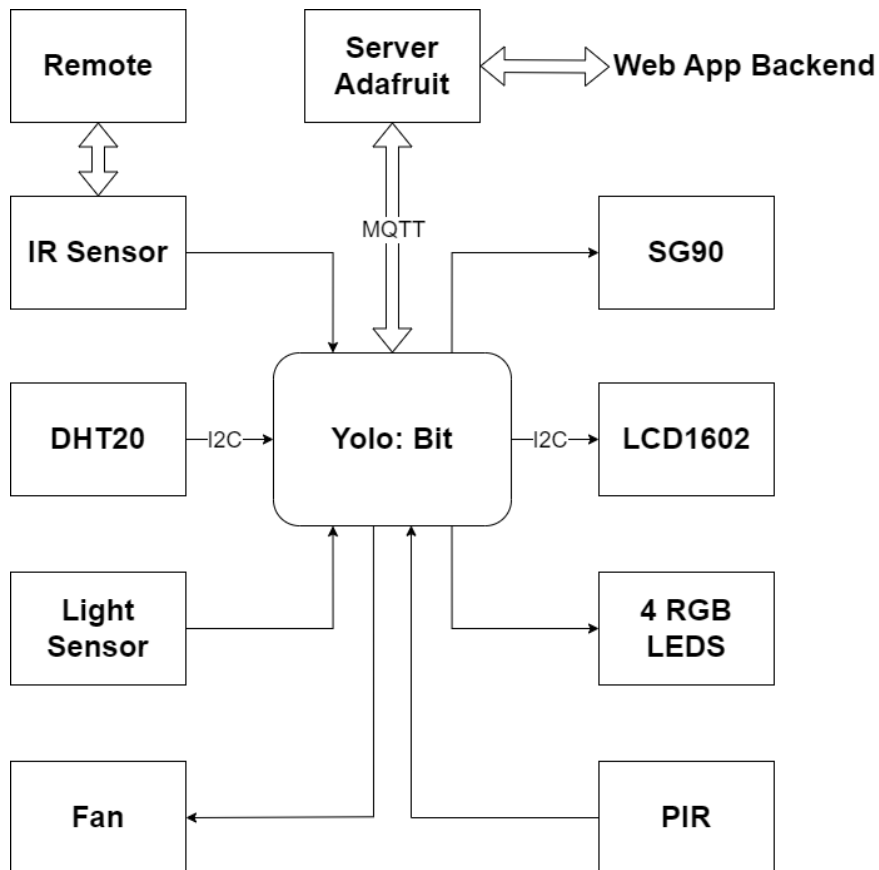
Chức năng Environment Monitoring sẽ hiển thị các dữ liệu đo đạc được từ môi trường. Chức năng này sẽ nhận các dữ liệu từ Server Adafruit thông qua giao thức MQTT, các dữ liệu đó được hệ thống phần cứng đo đạc và gửi lên Server Adafruit thông qua giao thức MQTT. Ngoài ra, hệ thống phần cứng cũng sẽ nhận các tín hiệu bật/tắt theo nhu cầu hoặc theo chế độ tự động từ web hệ thống gửi qua Server Adafruit.

Khi dữ liệu từ môi trường được gửi cho hệ thống chính, dữ liệu đó sẽ được lưu vào database. Các dữ liệu này sẽ được truy xuất và hiển thị khi người dùng sử dụng tính năng Environment Monitoring.

Password Lockage là tính năng mở cửa bằng mật khẩu trên trang web. Hệ thống sẽ kiểm tra mật khẩu người dùng nhập vào, sau đó gửi tín hiệu mở cửa cho Server Adafruit nếu người dùng nhập đúng mật khẩu.

Các tính năng Voice Command và Face Identification sẽ nhận một input đầu vào là một câu lệnh bằng giọng nói hoặc hình ảnh hiện tại từ camera thu được, hệ thống sau đó sẽ tiến hành kiểm tra các điều kiện cần thiết và gửi đi dữ liệu tính toán được cho Adafruit. Hệ thống phần cứng sau đó nhận các dữ liệu này và tiến hành kích hoạt các thiết bị.

4.2 Hệ thống phần cứng



Hình 9: Sơ đồ hệ thống các module phần cứng

Các module như DHT20 và LCD1602 sử dụng giao thức I2C để truyền và nhận dữ liệu từ Yolo:Bit.

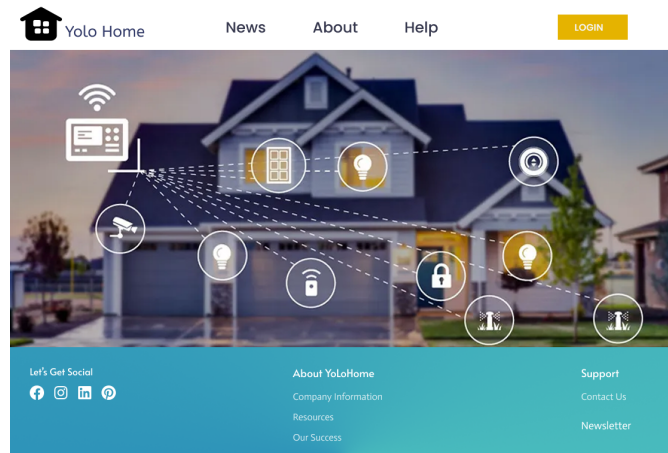
Yolo: Bit sẽ giao tiếp với Server Adafruit sử dụng giao thức MQTT.

Các module còn lại giao tiếp với Yolo: Bit sử dụng các kiểu kết nối Digital, Analog hoặc PWM. Remote sẽ giao tiếp với Mắt IR thông qua các tia hồng ngoại phát ra từ Remote.

4.3 Phần mềm

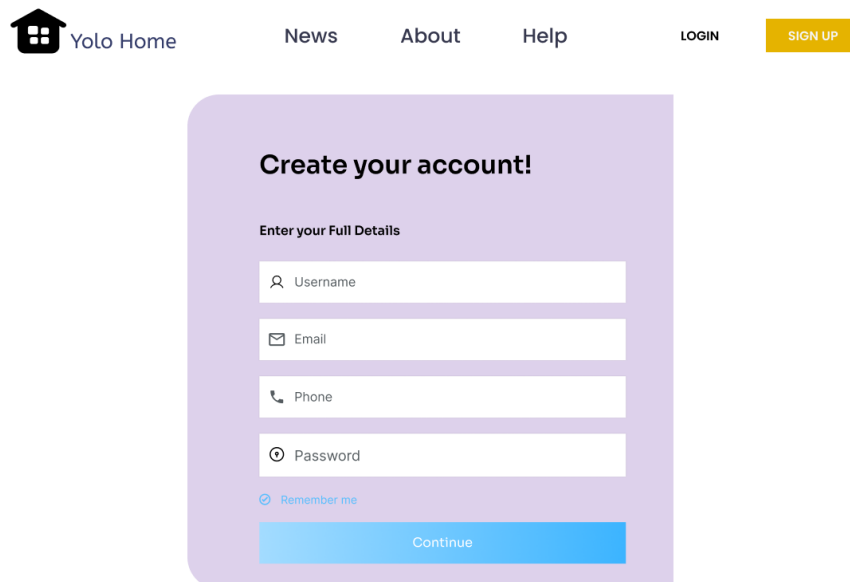
4.3.1 Thiết kế UI/UX

4.3.1.1 Trang bắt đầu



Hình 10: Figma Trang bắt đầu

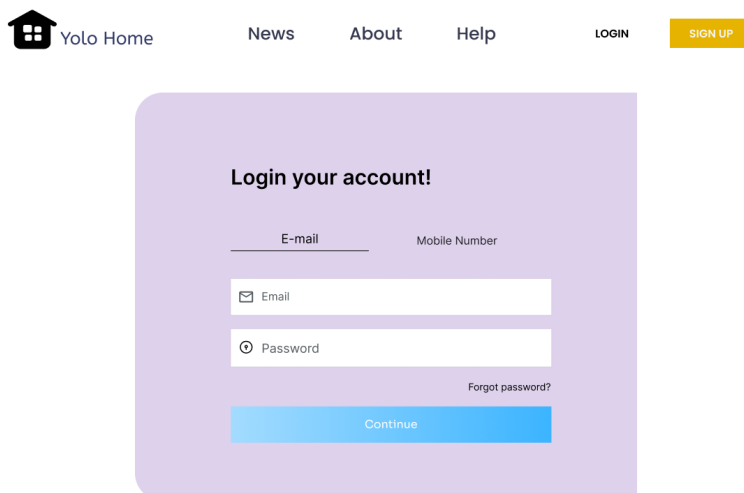
4.3.1.2 Trang đăng ký



Hình 11: Figma Trang đăng ký

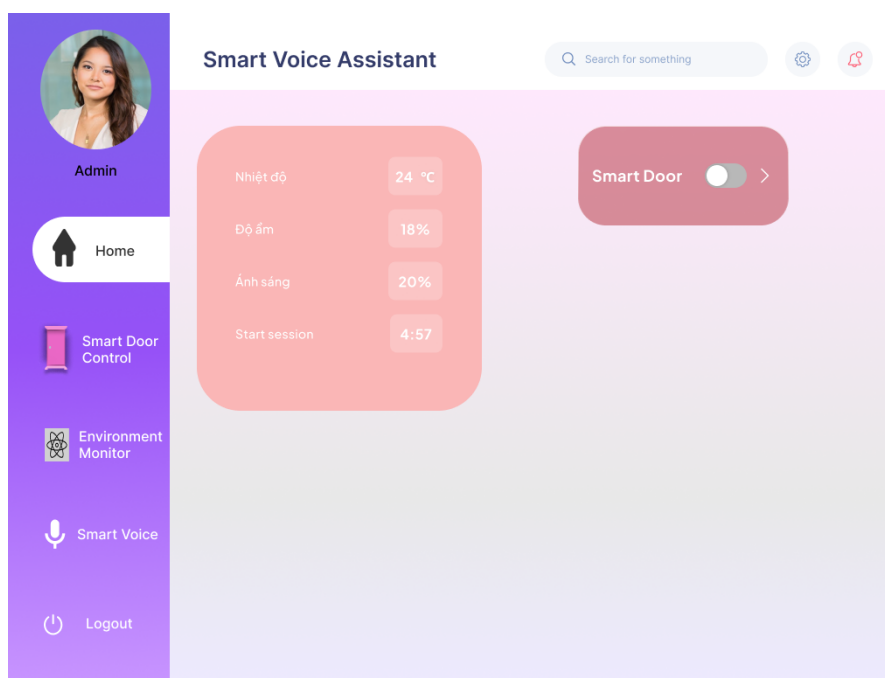


4.3.1.3 Trang đăng nhập



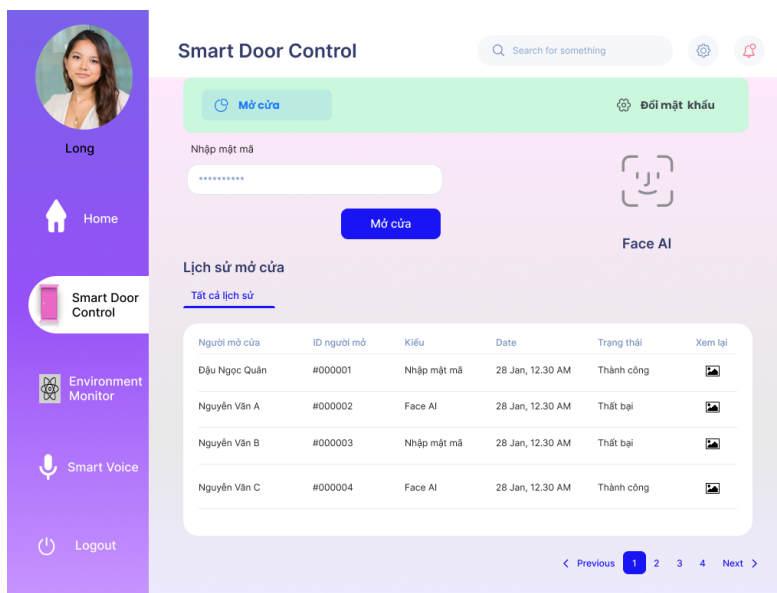
Hình 12: Figma Trang đăng nhập

4.3.1.4 Trang chủ



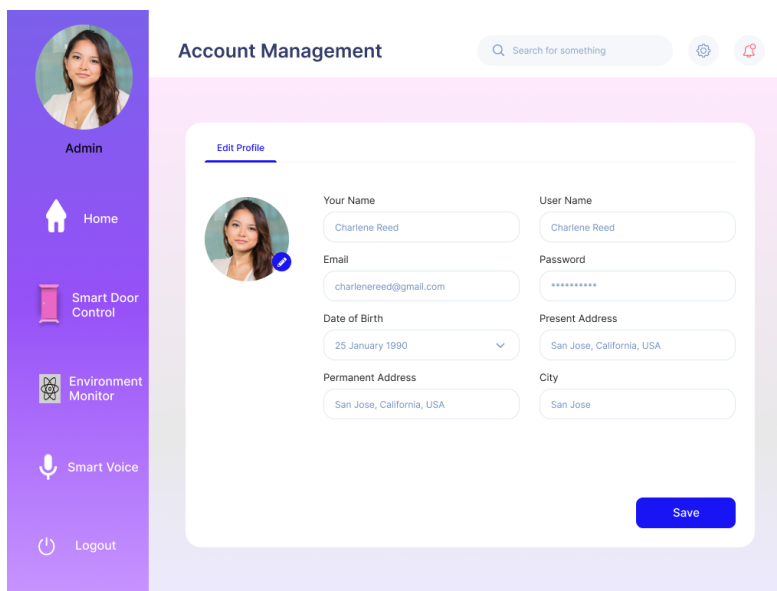
Hình 13: Figma Trang chủ

4.3.1.5 Hệ thống cửa



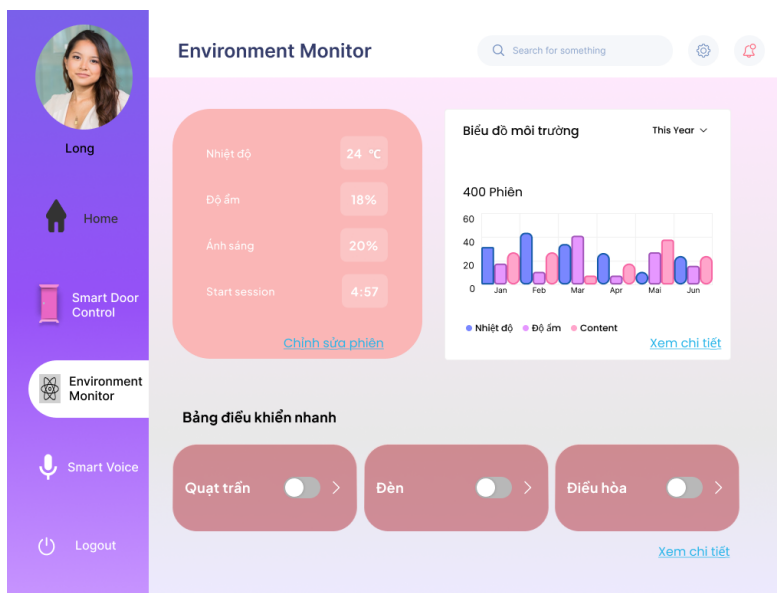
Hình 14: Figma Hệ thống cửa

4.3.1.6 Thông tin cá nhân



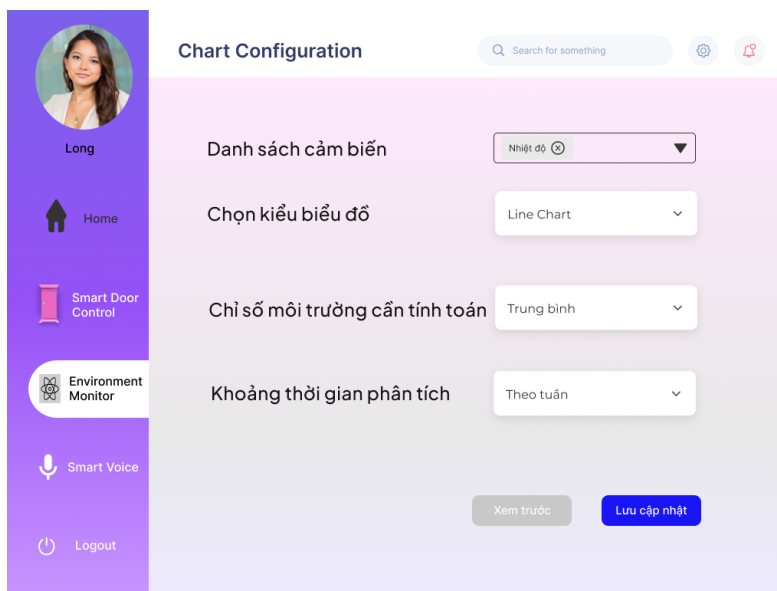
Hình 15: Figma Thông tin cá nhân

4.3.1.7 Giám sát môi trường



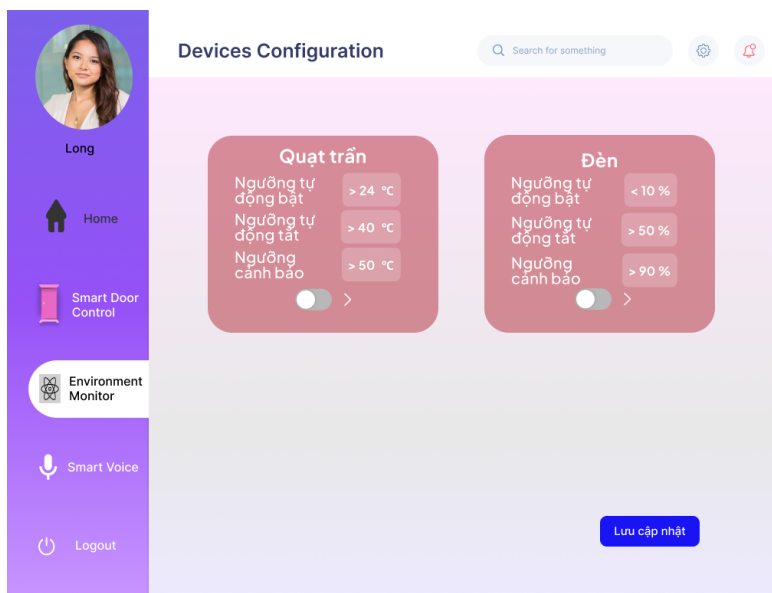
Hình 16: Figma Giám sát môi trường

4.3.1.8 Cấu hình biểu đồ



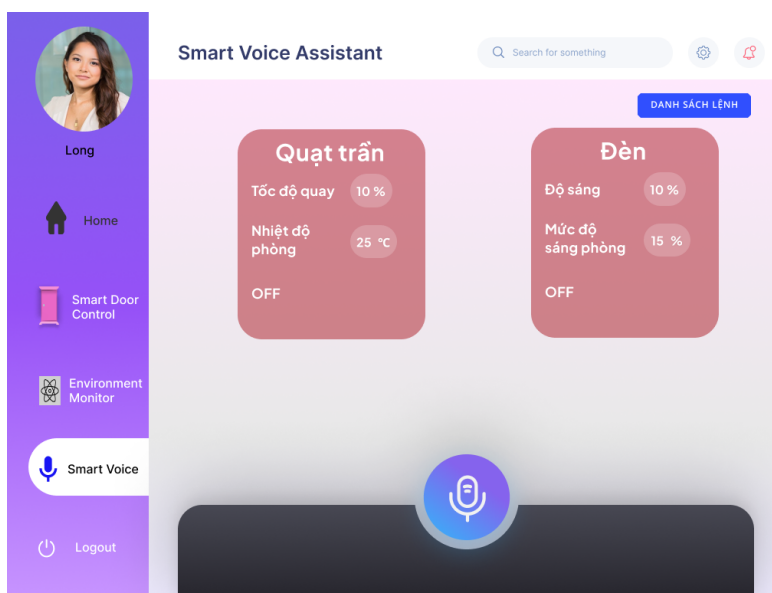
Hình 17: Figma Cấu hình biểu đồ

4.3.1.9 Cấu hình thiết bị



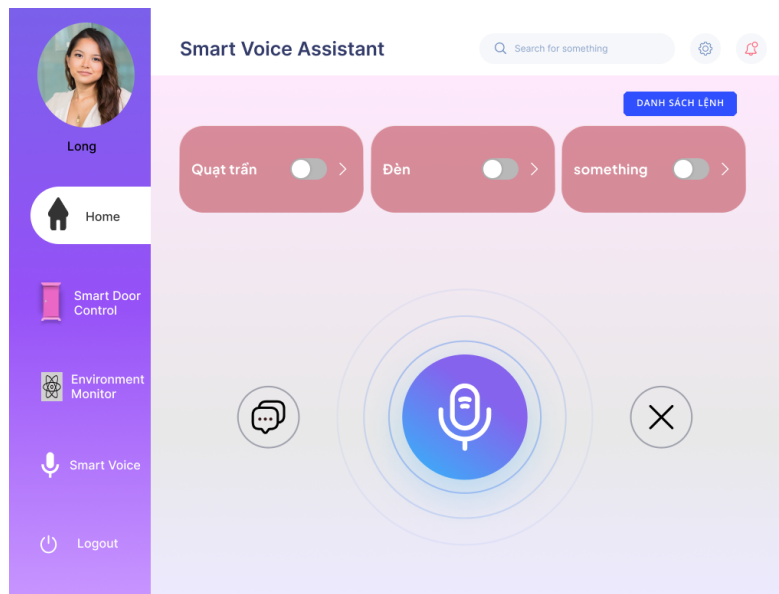
Hình 18: Figma Cấu hình thiết bị

4.3.1.10 Smart Voice 1



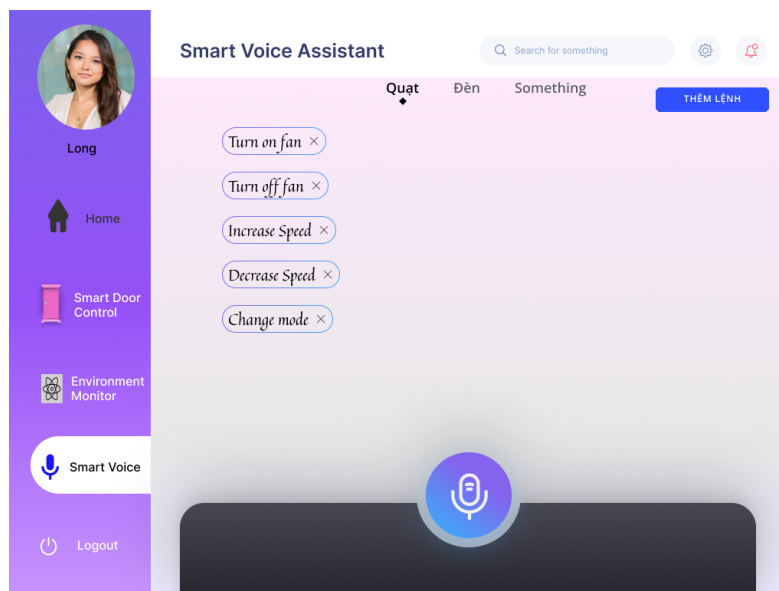
Hình 19: Figma Smart Voice 1

4.3.1.11 Smart Voice 2



Hình 20: Figma Smart Voice 2

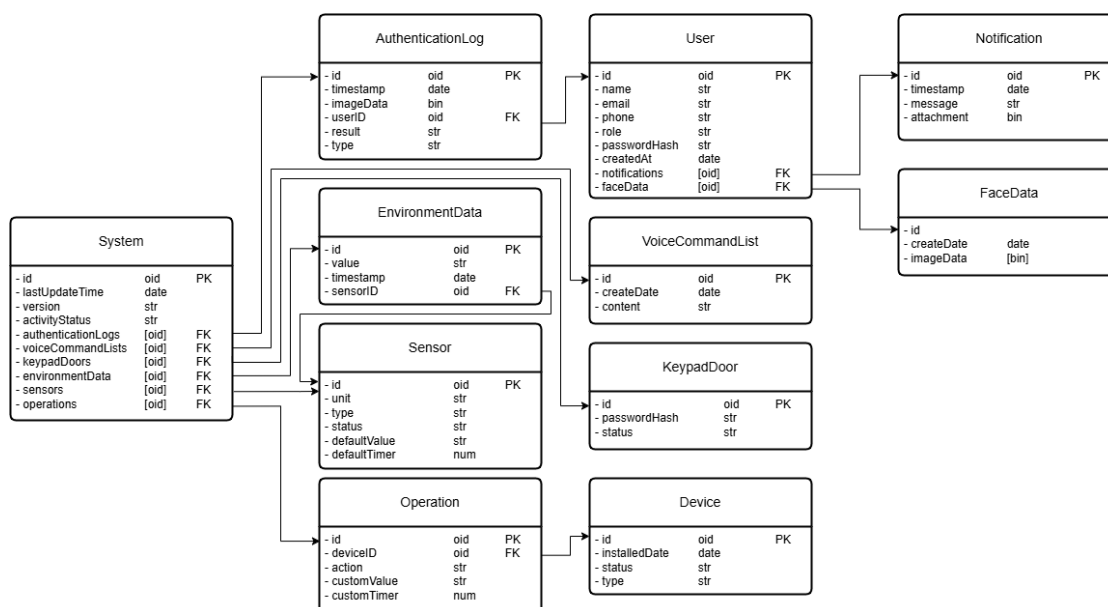
4.3.1.12 Danh sách lệnh



Hình 21: Figma Danh sách lệnh

4.3.2 Database

4.3.2.1 Document Schema Diagram



Hình 22: Document Schema Diagram của hệ thống

4.3.2.2 Mô tả các kiểu thực thể, các thuộc tính và mối liên kết

User (USER)

Mô tả: Là người sử dụng hệ thống với các thông tin cơ bản.

Thuộc tính:

- UserID (PK)
- PasswordHash
- Name
- Email
- Phone
- Role (Guest/Admin)
- CreatedAt

Liên kết:



- Một **User** có thể sở hữu nhiều **Device**.
- Một **User** có thể nhận nhiều **Notification**.
- Một **User** có thể tương tác với nhiều **System**.

Device (DEVICE)

Mô tả: Đại diện cho các thiết bị được quản lý bởi hệ thống.

Thuộc tính:

- DeviceID (PK)
- InstalledDate
- Status (On/Off)
- Type

Liên kết:

- Một **Device** thuộc quyền sở hữu của một **User**.
- Một **Device** được điều khiển bởi một hoặc nhiều **System**.
- Một **Device** có thể chứa **FaceData** để lưu trữ hình ảnh khuôn mặt.

FaceData (FACEDATA)

Mô tả: Lưu trữ thông tin khuôn mặt của người dùng.

Thuộc tính:

- FaceID (PK)
- ImageData
- CreatedDate

Liên kết:

- Một **FaceData** thuộc về đúng một **Device**.

System (SYSTEM)

Mô tả: Là hệ thống chính điều khiển các thiết bị, quản lý dữ liệu.

Thuộc tính:

- SystemID (PK)
- LastUpdateTime
- Version
- ActivityStatus

Liên kết:

- Điều khiển nhiều **Device**.
- Tạo ra nhiều **AuthenticationLog**.
- Quản lý nhiều **KeypadDoor** và chứa nhiều **VoiceCommandList**.
- Xử lý dữ liệu từ nhiều **EnviromentData**.
- Cấu hình nhiều **Sensor** và **Operation**.

AuthenticationLog (AUTHENTICATION_LOG)

Mô tả: Lưu lịch sử xác thực người dùng thông qua hệ thống.

Thuộc tính:

- LogID (PK)
- ImageData (có thể null)
- UserID (null nếu là người lạ)
- Timestamp
- Result
- Type

Liên kết:

- Một **AuthenticationLog** được tạo bởi một **System**.

VoiceCommandList (VOICE_COMMAND_LIST)

Mô tả: Danh sách các lệnh giọng nói điều khiển thiết bị.

Thuộc tính:

- CommandID (PK)
- CreatedDate
- Content

Liên kết:

- Một **VoiceCommandList** thuộc về một **System**.

KeypadDoor (KEYPAD_DOOR)

Mô tả: Đại diện cho các cửa khóa mã số trong hệ thống.

Thuộc tính:

- DoorID (PK)



- PasswordHash
- Status

Liên kết:

- Một **KeypadDoor** được quản lý bởi một **System**.

Notification (NOTIFICATION)

Mô tả: Lưu trữ thông báo cho người dùng.

Thuộc tính:

- NotificationID (PK)
- Timestamp
- Attachment
- Message

Liên kết:

- Một **Notification** được gửi đến một **User**.

EnviromentData (ENVIRONMENT_DATA)

Mô tả: Ghi nhận dữ liệu môi trường (nhiệt độ, độ ẩm, âm thanh, ...).

Thuộc tính:

- DataID (PK)
- Value
- Timestamp

Liên kết:

- Một **EnviromentData** được ghi nhận bởi một **Sensor**.
- Một **EnviromentData** được xử lý bởi một **System**.

Sensor (SENSOR)

Mô tả: Đại diện cho các cảm biến đo lường môi trường.

Thuộc tính:

- SensorID (PK)
- DefaultTimer
- DefaultValue
- Type (Mic, nhiệt, ánh sáng,...)

- Status
- Unit

Liên kết:

- Một **Sensor** ghi nhận nhiều **EnviromentData**.
- Một **Sensor** được cấu hình bởi một **System**.

4.3.2.3 Hiện thực Database

Nhóm quyết định chọn MongoDB để hiện thực Database vì:

- 1. Mô hình dữ liệu linh hoạt (Flexible Schema):** MongoDB là cơ sở dữ liệu NoSQL dạng Document-based, cho phép lưu trữ dữ liệu dưới dạng JSON (hoặc BSON) linh hoạt. Điều này cho phép các thực thể như **User**, **Device**, **System**, **Notification**,... có thể được lưu trữ dưới dạng Document mà không cần tuân thủ một cấu trúc cứng nhắc như trong các cơ sở dữ liệu quan hệ (RDBMS). Mỗi document có thể có các thuộc tính khác nhau, phù hợp với sự thay đổi của hệ thống theo thời gian.
- 2. Hiệu năng cao khi truy vấn dữ liệu phức tạp:** MongoDB cho phép sử dụng Embedded Document hoặc Reference Document để tối ưu hóa các truy vấn liên quan:
 - Với các quan hệ 1-N, như **User** sở hữu nhiều **Device** hoặc **Notification**, ta có thể lưu trực tiếp danh sách ID hoặc document của **Device**, **Notification** vào document của **User** mà không cần phải join bảng như RDBMS.
 - Với các quan hệ N-M, như **System** quản lý nhiều **KeypadDoor** và **VoiceCommandList**, MongoDB cung cấp khả năng lưu trữ ID tham chiếu hoặc array trực tiếp, giúp tối ưu hóa tốc độ truy vấn.
- 3. Khả năng mở rộng tốt (Horizontal Scaling):** MongoDB hỗ trợ sharding – phương thức phân tán dữ liệu theo chiều ngang (horizontal scaling). Điều này giúp hệ thống dễ dàng mở rộng khi số lượng thiết bị (**Device**), dữ liệu môi trường (**EnviromentData**), hay thông tin đăng nhập (**AuthenticationLog**) gia tăng.
- 4. Hỗ trợ lưu trữ dữ liệu dạng đa phương tiện:** Trong ERD, các thông tin như **ImageData** trong **FaceData** hoặc các tập tin đính kèm trong **Notification** là hình ảnh và tập tin. MongoDB hỗ trợ **GridFS**, cho phép lưu trữ và truy xuất tập tin một cách hiệu quả.
- 5. Truy vấn dễ dàng với Aggregation Framework:** MongoDB cung cấp **Aggregation Pipeline** giúp thực hiện các phép lọc, nhóm, tính toán phức tạp trên dữ liệu một cách dễ dàng, tối ưu hơn so với các câu truy vấn SQL phức tạp.
- 6. Tính tương thích cao với các công nghệ hiện đại:** MongoDB tương thích tốt với các công nghệ web như Node.js, Express.js và các microservices. Điều này giúp dễ dàng xây dựng API giao tiếp giữa hệ thống nhận diện khuôn mặt, cảm biến môi trường và thiết bị IoT.
- 7. Thích hợp cho hệ thống IoT và Real-time:** Mô hình ERD trên có nhiều thiết bị **Device**, cảm biến **Sensor** và dữ liệu môi trường **EnviromentData** cập nhật liên tục.



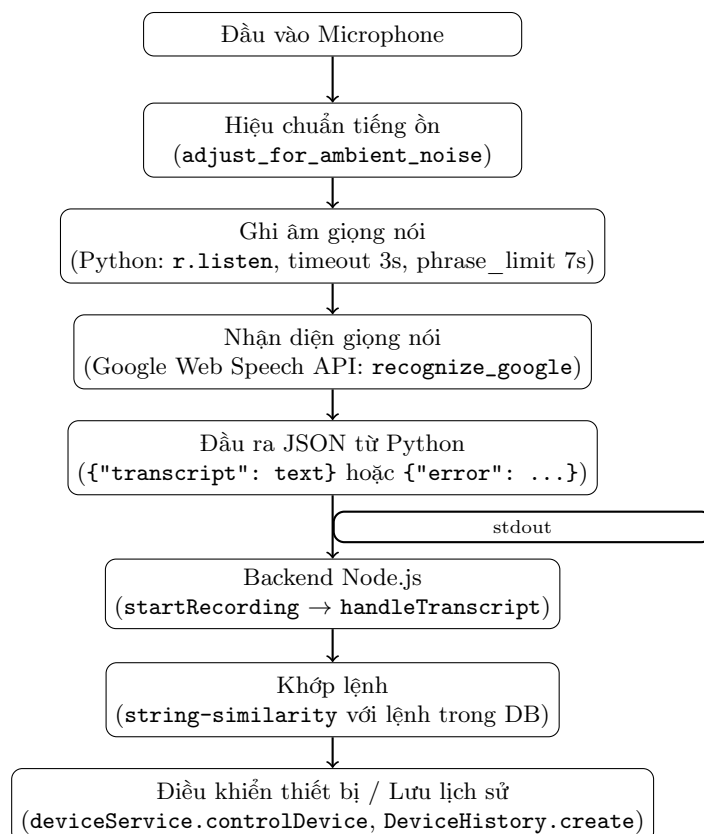
MongoDB hỗ trợ cập nhật theo thời gian thực, giúp hệ thống IoT hoạt động mượt mà hơn.

4.4 Phần trí tuệ nhân tạo

4.4.1 Nhận diện giọng nói

1. Luồng xử lý (Pipeline AI)

Quy trình xử lý giọng nói từ đầu vào micro đến khi ra lệnh điều khiển được thể hiện như sau:



Hình 23: Luồng xử lý chính khi nhận diện giọng nói và điều khiển thiết bị thành công.

2. Thiết kế dữ liệu

- **Raw Audio (đầu vào cho Python):** Dữ liệu âm thanh được thu trực tiếp từ microphone bởi thư viện `speech_recognition`.
- **Transcript (đầu ra từ Python, đầu vào cho Node.js):** Chuỗi ký tự UTF-8 tiếng Việt sau khi nhận diện. Ví dụ: "mở đèn".
- **Command Data (trong Database, sử dụng bởi Node.js):** Các câu lệnh mẫu (ví dụ: "mở đèn", "tắt quạt") được lưu trữ để so khớp với transcript.
- **Device Control Payload (gửi tới thiết bị):** Dữ liệu điều khiển, ví dụ "1" để bật đèn, "0" để tắt đèn.

3. Cấu hình tham số chính (trong mã nguồn mic_stream.py và handleTranscript.js)

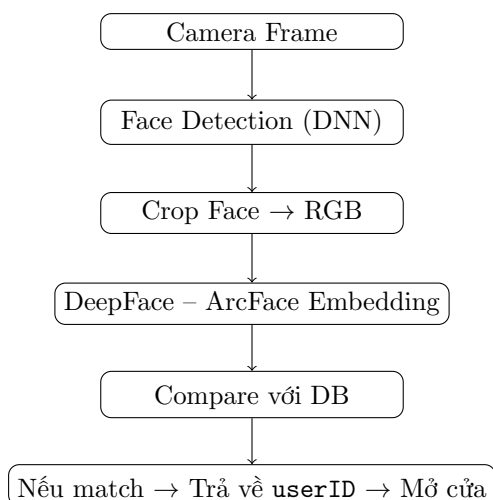
Tham số	Ý nghĩa	Giá trị (từ code)
CALIB_DURATION	Thời gian hiệu chuẩn tiếng ồn	1 giây
TIMEOUT	Giới hạn thời gian chờ bắt đầu nói	3 giây
PHRASE_LIMIT	Giới hạn thời gian ghi âm tối đa	7 giây
LANG	Ngôn ngữ nhận diện	'vi-VN'
API_ENGINE	Engine nhận diện	'google'
THRESHOLD (JS)	Ngưỡng tương đồng để khớp lệnh	0.5

4. Giao tiếp giữa các thành phần

- Script Python (mic_stream.py) thực hiện thu âm và gọi API nhận diện giọng nói. Kết quả được xuất ra `stdout` dưới dạng chuỗi JSON.
- Backend Node.js (startRecording trong controller) gọi thực thi script Python này như một tiến trình con (child_process.spawn).
- Node.js nhận dữ liệu JSON từ `stdout` của Python, sau đó chuyển cho hàm `voiceController.handleTranscript` để xử lý.
- Hàm `handleTranscript` so khớp transcript với các lệnh đã định nghĩa trong cơ sở dữ liệu bằng `string-similarity`.
- Dựa trên lệnh khớp nhất (nếu vượt ngưỡng `THRESHOLD`), `deviceService.controlDevice` được gọi để gửi lệnh điều khiển tới thiết bị vật lý. Lịch sử hành động (thành công hoặc thất bại) và các thông báo lỗi (nếu có) được lưu vào cơ sở dữ liệu.

4.4.2 Nhận diện khuôn mặt

1. Luồng xử lý (Pipeline AI)



2. Thiết kế dữ liệu

- **Embeddings:** ma trận $N \times 512$, với N là số người dùng đã đăng ký.
- **Labels:** danh sách chuỗi `userID` tương ứng với từng vector embedding.
- Cấu trúc dữ liệu đơn giản, dễ load và xử lý nhanh bằng NumPy.

3. Cấu hình tham số chính

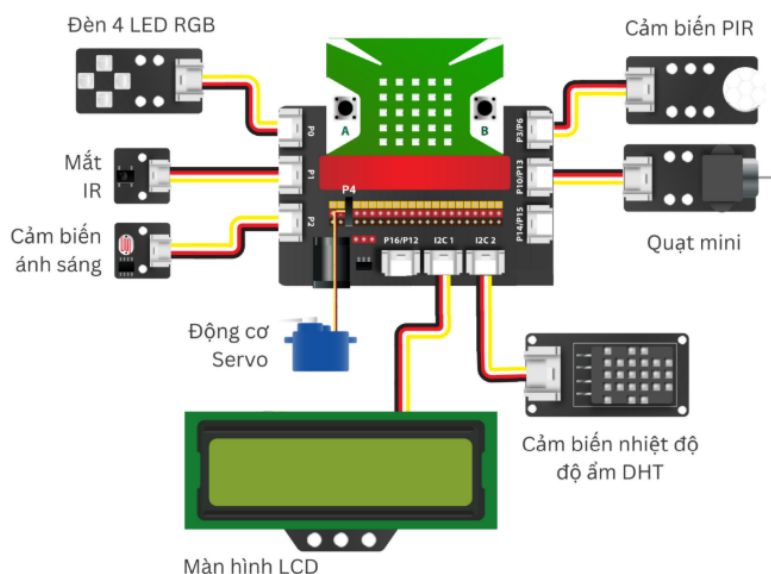
Tham số	Ý nghĩa	Giá trị đề xuất
THRESHOLD	Ngưỡng khoảng cách Euclidean để xác nhận	10.0
SKIP_FRAMES	Số frame bỏ qua giữa các lần nhận diện	10
DELAY_AFTER_MATCH	Thời gian chờ sau khi nhận diện thành công	5 giây

4. Giao tiếp với hệ thống

- Script Python hoạt động như một module xử lý độc lập.
- Giao tiếp với backend web thông qua:
 - `stdin/stdout` với định dạng JSON
- Cho phép tách rời phần giao diện và phần AI, để bảo trì và thay thế mô hình sau này.

5 Hiện thực

5.1 Hệ thống phần cứng



Hình 24: Kết nối phần cứng

Đối tượng	Kênh thông tin	Kiểu dữ liệu
Nhiệt độ	Temperature	Float
Độ ẩm	Humidity	Float
Cường độ ánh sáng	Light Intensity	Integer
Khóa/Mở khóa cửa	Door	Boolean
Bật/tắt đèn	Light	Boolean
Bật/tắt quạt	Fan	Boolean

Bảng 1: Kênh thông tin trên Adafruit cho các đối tượng

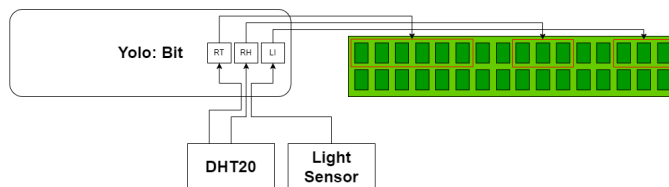
5.1.1 Màn hình LCD1602, DHT20 và Thời gian

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	2	5	.	7	*	C			6	3	%			2	1	%
1	2	1	/	1	0	/	2	0	2	2		1	3	:	3	8

Hình 25: Màn hình hiển thị mẫu của LCD 1602

Hình 25 là ví dụ để hiển thị trên LCD 1602 các thông số đo được, với dòng đầu tiên lần lượt là Nhiệt độ, độ ẩm và ánh sáng, dòng thứ 2 sẽ hiển thị Ngày, Giờ và Phút. Thời gian cập nhật các

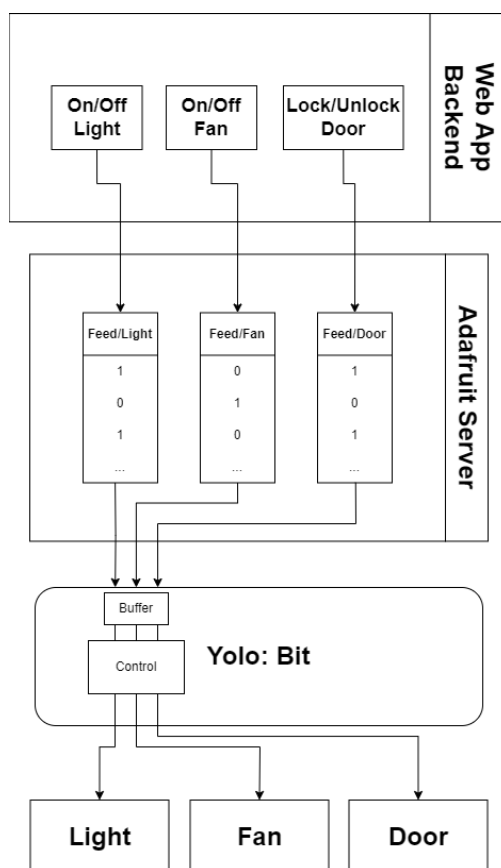
giá trị nhiệt độ, độ ẩm và ánh sáng sẽ là 5 giây, còn Ngày, Giờ và Phút sẽ là 30 giây.



Hình 26: Luồng thực thi của LCD 1602

Hình 26 thể hiện luồng hiển thị của màn hình LCD1602, các dữ liệu đo đạc được từ các cảm biến sẽ được lưu vào các biến RT(Read Temperature), RH(Read Humidity) và LI(Light Intensity). Sau đó sẽ được gửi cho màn hình LCD1602 và hiển thị các thông số đo được ở các vị trí đã setup trước.

5.1.2 Điều khiển các thiết bị thủ công

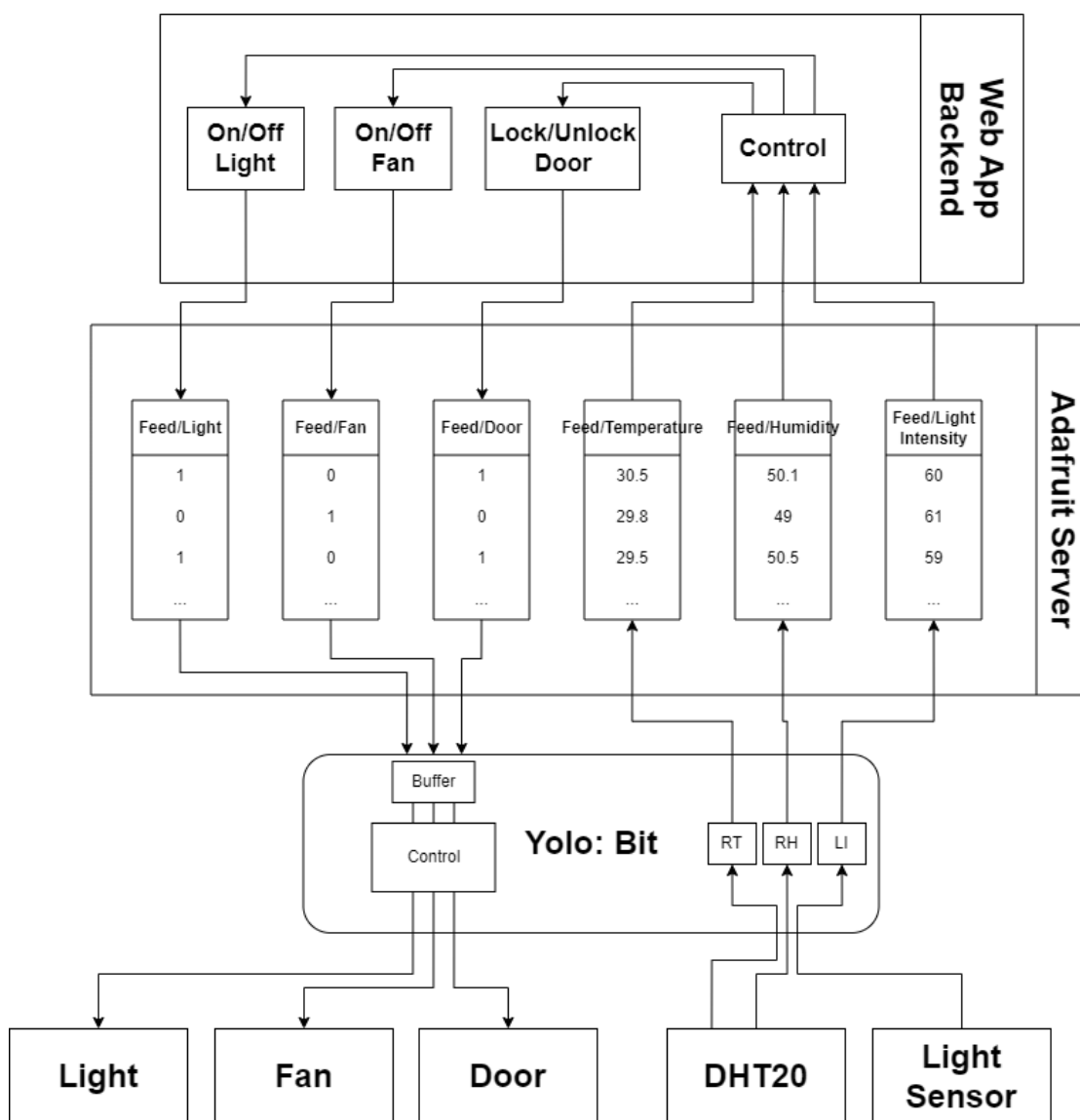


Hình 27: Luồng điều khiển các thiết bị qua giao diện Web App

Ở trang điều khiển các thiết bị trên Web App, khi người dùng sử dụng giao diện Bật/Tắt Đèn, Bật/Tắt Quạt hay Khóa/Mở khóa Cửa, tín hiệu sẽ được gửi cho các kênh trên Adafruit Server tương ứng như ở Bảng 1.

Yolo: Bit có một tính năng đặc biệt là lưu về giá trị trên các kênh khi các kênh đó cập nhật giá trị mới. Nếu trên các kênh đó có giá trị mới, mạch Yolo: Bit sẽ lập tức tải về các giá trị đó và lưu vào Buffer. Các giá trị tải về tương ứng với các kênh sẽ có một Buffer riêng, các Buffer này sau đó sẽ được khối Control thu thập và gửi các tín hiệu điều khiển cho các thiết bị tương ứng.

5.1.3 Tự động bật/tắt các thiết bị



Hình 28: Luồng thực thi của chế độ tự động



Chế độ tự động sẽ được mở rộng từ chế độ "Điều khiển các thiết bị thủ công". Web App Backend sẽ đọc các dữ liệu đo được từ cảm biến và tiến hành điều khiển các thiết bị trong nhà.

Luồng thực thi của chế độ tự động theo Hình 28 như sau, khi DHT20 hoặc Cảm biến ánh sáng thu thập dữ liệu từ môi trường, dữ liệu đó sẽ được gửi cho Vi điều khiển Yolo: Bit và lưu nó vào các biến RT(Read Temperature), RH(Read Humidity) và LI(Light Intensity). Sau đó, Yolo: Bit sẽ tiến hành gửi dữ liệu từ các biến này lên các kênh được định nghĩa tại Bảng 1 trên Adafruit Server sử dụng giao thức MQTT và lưu lại các dữ liệu đó. Web App Backend sẽ tiến hành thu thập các dữ liệu mới nhất từ các kênh và tiến hành so sánh các dữ liệu. Nếu dữ liệu rơi vào vùng kích hoạt thiết bị (chẳng hạn như bật quạt nếu nhiệt độ cao hơn 37 độ C) thì Web App Backend sẽ tiến hành gửi các tín hiệu điều khiển cho các khối "On/Off Light" và "On/Off Fan", các khối này sẽ tiến hành kích hoạt các thiết bị tương tự như ở chế độ "Điều khiển các thiết bị thủ công".

5.2 Phần mềm

Kiến trúc tổng quan

Hệ thống phần mềm được tổ chức thành các module chính để đảm bảo tính mở rộng, bảo trì và dễ tích hợp với phần cứng:

Các module chính

- **Auth Service:** Xác thực và cấp JWT.
- **User Service:** Quản lý profile, cập nhật avatar và thông tin cá nhân.
- **Door Control Service:** Mở cửa bằng mật khẩu hoặc FaceAI; cập nhật khuôn mặt; quản lý lịch sử; điều khiển qua MQTT.
- **Environment Service:** Nhận dữ liệu cảm biến qua MQTT; điều chỉnh ngưỡng; bật/tắt thiết bị tự động.
- **Voice Assistant Service:** Nhận lệnh giọng nói (Web Speech API hoặc microphone), parse lệnh, gửi yêu cầu.
- **Notification Service:** Gửi và lưu thông báo.

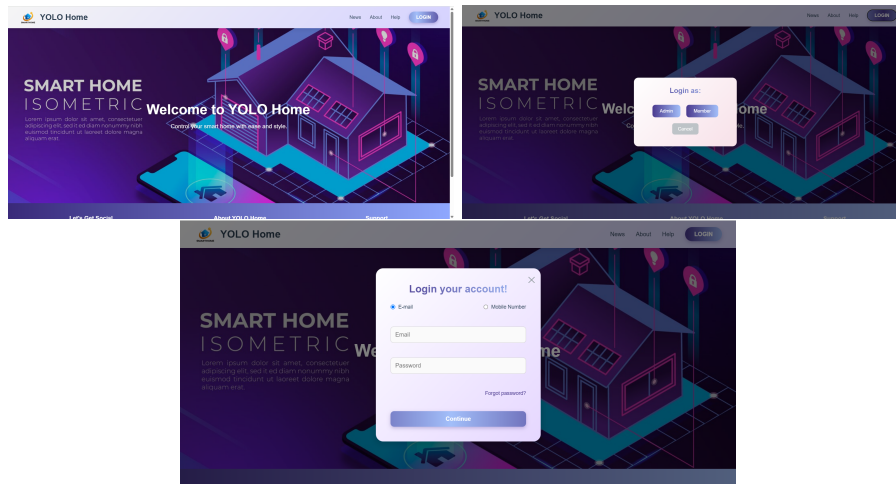
Luồng dữ liệu

1. UI gọi REST API → Backend xác thực JWT → chuyển request đến module tương ứng.
2. Với FaceAI: Backend spawn subprocess Python → nhận kết quả → trả về client.
3. Với lệnh giọng nói: audio từ frontend → Voice Assistant Service xử lý STT → thực thi lệnh qua REST hoặc MQTT.
4. Với IoT: Backend publish/subscribe MQTT để cập nhật trạng thái và điều khiển thiết bị.

Luồng hoạt động phần mềm trong thực tế

1. Đăng nhập tài khoản

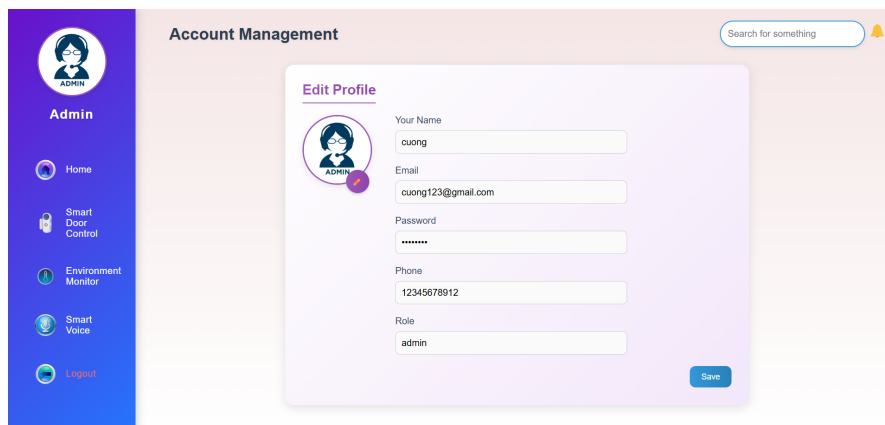
- Người dùng nhập email và mật khẩu.
- Frontend gửi request POST /admin/login hoặc POST /member/login tùy vào vai trò.
- Backend xác thực, trả về JWT.
- Frontend lưu token và chuyển đến dashboard.



Hình 29: Luồng đăng nhập: (1) Trang đăng nhập, (2) Chọn role, (3) Điền thông tin

2. Cập nhật thông tin người dùng

- Người dùng vào trang Profile.
- Chỉnh sửa avatar, tên, email, nhấn “Lưu”.
- Frontend gửi PUT /user/updateUser.
- Backend cập nhật và trả về profile mới.

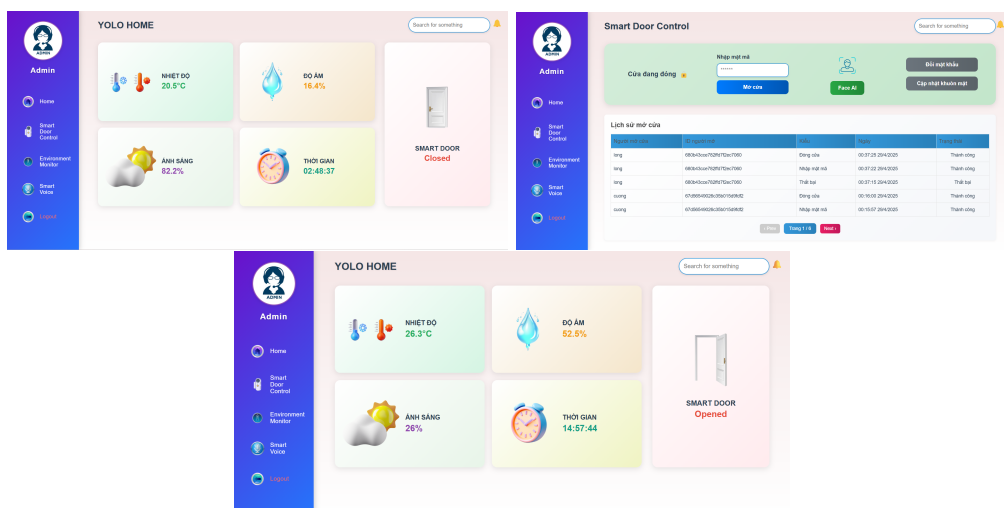


Hình 30: Giao diện cập nhật thông tin người dùng

3. Mở cửa bằng mật mã

- Người dùng nhập mã trên giao diện hoặc bàn phím số.

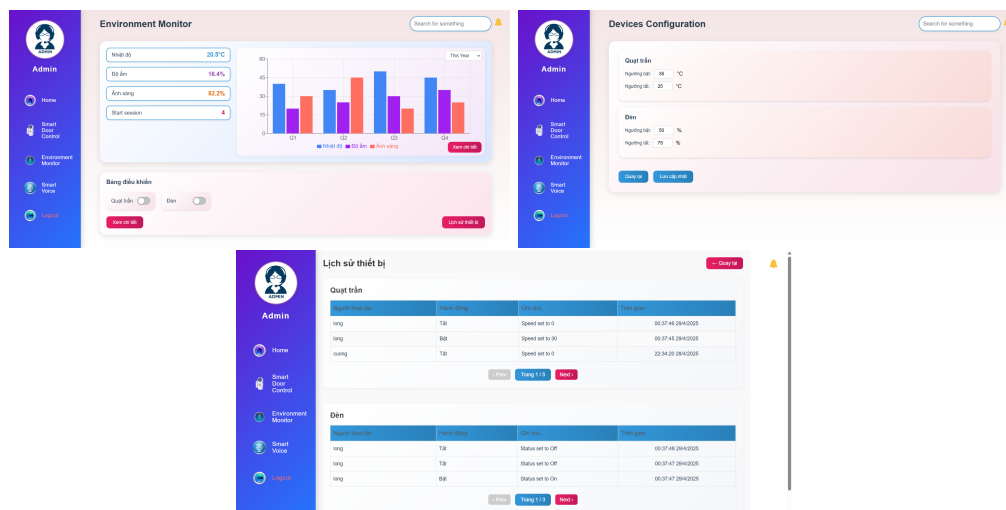
- Frontend gửi POST `/smart_door/access` kèm mã và `userId`.
- Backend kiểm tra mã, nếu đúng publish lệnh qua MQTT.
- Bộ điều khiển relay nhận lệnh, mở cửa.



Hình 31: Luồng mở cửa bằng mật mã: (1) Trang chủ, (2) Nhập mã, (3) Cửa mở

4. Xem thông tin môi trường

- Thiết bị IoT gửi dữ liệu cảm biến qua MQTT.
- Backend subscribe, lưu vào MongoDB.
- Frontend lấy dữ liệu và hiển thị.
- Khi kích hoạt thiết bị sẽ lưu lại thông tin truy cập.



Hình 32: Giao diện giám sát môi trường: (1) Thông tin môi trường, (2) Cập nhật threshold cho môi trường, (3) Lưu lịch sử truy cập

5.3 Phần trí tuệ nhân tạo

5.3.1 Nhận diện giọng nói

1. Môi trường triển khai

- Ngôn ngữ: Python 3 (cho module nhận diện giọng nói), Node.js (cho backend và điều phối).
- Thư viện Python chính: `SpeechRecognition` (phiên bản tương thích với `recognize_google`).
- Thư viện Node.js chính: `child_process` (để chạy script Python), `string-similarity` (để so khớp lệnh), `mongoose` (tương tác MongoDB), các thư viện liên quan đến Express.js và MQTT (nếu `deviceService` sử dụng).
- Thiết bị: Micro tích hợp của laptop hoặc micro rời (USB) để thu âm.

2. Mô tả mã nguồn chính Hệ thống nhận diện và xử lý lệnh giọng nói bao gồm:

- `mic_stream.py` (Python):
 - Hiệu chuẩn nhiễu nền từ microphone.
 - Ghi âm một đoạn giọng nói (tối đa 7 giây sau khi phát hiện tiếng nói, chờ tối đa 3 giây để bắt đầu nói).
 - Gửi dữ liệu âm thanh tới Google Web Speech API để nhận diện.
 - In kết quả (transcript hoặc lỗi) ra `stdout` dưới dạng JSON.
- `voice.controller.js` (Node.js - chứa `handleTranscript`):
 - Nhận đầu vào là văn bản `transcript` từ script Python.
 - Truy vấn danh sách các lệnh mẫu từ cơ sở dữ liệu (`commandController.getCommandsFromDB()`).
 - Sử dụng `string-similarity.compareTwoStrings` để tìm lệnh mẫu gần giống nhất với `transcript`.
 - Nếu độ tương đồng vượt ngưỡng (ví dụ 0.5), thực hiện hành động tương ứng (bật/tắt đèn, quạt) thông qua `deviceService.controlDevice`.
 - Cập nhật trạng thái thiết bị và lưu lịch sử hành động vào database (`Device.findOneAndUpdate`, `DeviceHistory.create`).
 - Nếu không khớp lệnh hoặc có lỗi, ghi log, lưu lịch sử (với trạng thái thất bại) và có thể tạo thông báo (`Notification.create`).
- Route handler (Node.js - chứa `startRecording`):
 - Tiếp nhận yêu cầu HTTP (ví dụ: từ client).
 - Gọi script `mic_stream.py` bằng `spawn`.

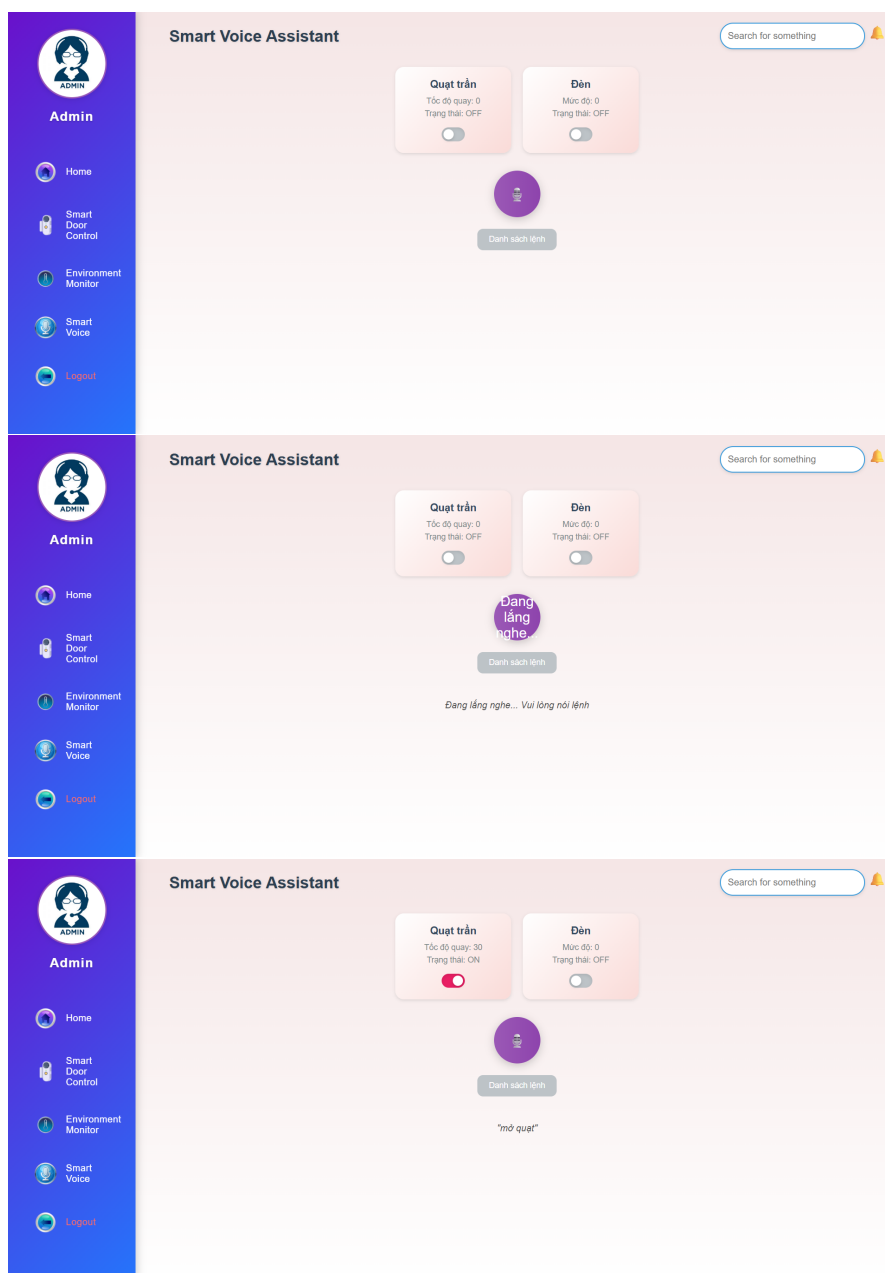
- Thu nhận output từ Python, parse JSON và gọi `voiceController.handleTranscript`.
- Trả về phản hồi HTTP cho client.

3. Tích hợp với hệ thống điều khiển nhà thông minh

- Giao diện người dùng kích hoạt chức năng ghi âm qua một API endpoint trên backend Node.js.
- Backend Node.js gọi script Python `mic_stream.py` để thu thập và nhận diện giọng nói.
- Transcript nhận được sẽ được `handleTranscript` xử lý để xác định ý định người dùng.
- `deviceService` trong backend sẽ gửi lệnh điều khiển đến các thiết bị thông minh (ví dụ: qua MQTT, HTTP requests đến gateway của thiết bị).
- Trạng thái thiết bị trong cơ sở dữ liệu được cập nhật, và lịch sử được ghi lại.

4. Kịch bản thử nghiệm thực tế

- Thử nghiệm với các tình huống điều khiển thiết bị cụ thể:
 - Câu lệnh rõ ràng (VD: "Mở đèn", "Tắt quạt") → Hệ thống nhận diện đúng transcript, `handleTranscript` khớp lệnh thành công, thiết bị được điều khiển và trạng thái được cập nhật.
 - Câu lệnh gần đúng (VD: "bật cái đèn lên") → Nếu `string-similarity` vẫn cho kết quả trên ngưỡng với lệnh "mở đèn", hệ thống vẫn xử lý đúng.
 - Câu lệnh không rõ ràng hoặc không có trong danh sách lệnh (VD: "trời hôm nay đẹp quá", nói ngắt quãng, có nhiều tiếng ồn) → `recognize_google` có thể trả về transcript không chính xác hoặc `UnknownValueError`. Nếu có transcript nhưng không khớp lệnh nào (dưới ngưỡng similarity), hệ thống ghi nhận vào `DeviceHistory` là "failed" và có thể tạo `Notification`.
 - Không có giọng nói (im lặng quá 3 giây) → `mic_stream.py` trả về lỗi "Không nghe bạn nói gì.", backend xử lý lỗi này và thông báo cho người dùng.
- Hệ thống cần phản hồi đúng logic trong các trường hợp trên, đảm bảo tính tin cậy và trải nghiệm người dùng tốt.



Hình 33: Minh họa quá trình kiểm thử thực tế.

5.3.2 Nhận diện khuôn mặt

1. Môi trường thực tế

- Ngôn ngữ: Python, sử dụng các thư viện OpenCV, DeepFace, NumPy.

- Thiết bị: Webcam tích hợp của laptop, xử lý real-time từng frame.

2. Mã nguồn triển khai Hệ thống gồm 2 script chính:

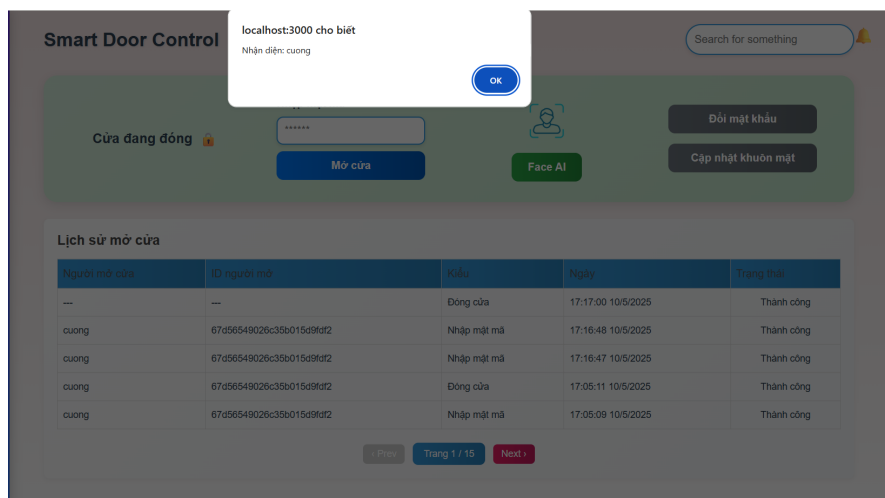
- `train_model_face.py`: dùng để tạo database người dùng.
 - Nhận dữ liệu từ file JSON (chứa ảnh base64 hoặc path ảnh).
 - Giải mã ảnh → phát hiện mặt → trích xuất embedding → lưu cùng userID.
- `recognize_camera.py`: dùng để nhận diện trong thời gian thực.
 - Mở webcam → đọc frame → detect + crop face.
 - Sinh embedding với ArcFace → so khớp với database.
 - Nếu khớp: trả về userID; ngược lại trả về "Unknown".

3. Tích hợp với hệ thống thực tế

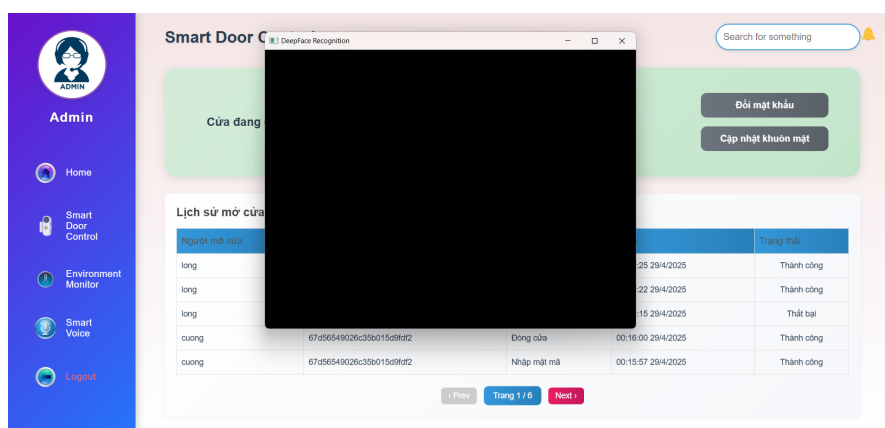
- Backend gọi script và truyền vào userID mục tiêu.
- Script xử lý và trả kết quả nhận diện qua `stdout` (dạng JSON).
- Nếu đúng người dùng: backend điều khiển mở cửa (qua GPIO, relay, v.v.).

4. Kết quả thử nghiệm thực tế

- Thử nghiệm với 2 tình huống:
 - Người thật đã đăng ký → hệ thống nhận diện đúng và mở cửa.
 - Người lạ chưa đăng ký hoặc Không có mặt người trong khung hình → hệ thống không phản hồi.
- Hệ thống xử lý đúng trong cả 2 trường hợp.



Hình 34: Minh họa quá trình kiểm thử thực tế: nhận diện đúng người dùng.



Hình 35: Minh họa quá trình kiểm thử thực tế: phát hiện người lạ hoặc không có mặt.

6 Tổng kết

6.1 Những gì đã làm được

- **Triển khai 4 module chính:**
 - *Giám sát môi trường*: thu thập thời gian thực các chỉ số (nhiệt độ, độ ẩm, ánh sáng, khí gas), hiển thị dashboard và lưu lịch sử.
 - *Mở cửa mật mã*: cơ chế nhập mã qua giao diện, giới hạn số lần sai, ghi log và điều khiển relay.
 - *Điều khiển bằng giọng nói*: tích hợp Google Speech-to-Text, xử lý lệnh với `string-similarity`, điều khiển thiết bị qua MQTT/REST, lưu lịch sử.
 - *Nhận diện khuôn mặt (FaceAI)*: sử dụng SSD + ArcFace, train và recognize real-time, spawn subprocess Python từ Node.js, mở cửa khi xác thực thành công.
- **Kiến trúc đa ngành, dễ mở rộng:**
 - Microservices rõ ràng: Auth, User, DoorControl, Environment, VoiceAssistant, Notification.
 - Frontend React + Redux; Backend Node.js/Express; MongoDB; MQTT cho IoT.
- **Tích hợp AI & IoT:**
 - Kết nối ESP32 (Yolo:Bit) với server.
 - AI hoạt động real-time, latency thấp.
- **Chất lượng và bảo mật:**
 - Xác thực JWT, phân quyền rõ ràng.
 - Cấu hình trong `.env` (DB_URL, JWT_SECRET, MQTT_BROKER_URL).
 - Logging chi tiết cho voice và camera.
- **Tài liệu và trình bày:**
 - Báo cáo đầy đủ với use-case, activity diagrams.
 - Prototype UI trên Figma, mockup rõ ràng.

6.2 Hướng cải thiện trong tương lai

- **Cải tiến AI:**
 - Triển khai mô hình nhận diện giọng nói cục bộ (ví dụ OpenAI Whisper fine-tuned).
 - Fine-tune ArcFace hoặc thử ResNet100 để tăng độ chính xác.



- **Mở rộng tính năng:**

- Ứng dụng di động (iOS/Android).
- Thêm sensor (chuyển động) nâng cao giám sát.

- **Hoàn thiện UX/UI:**

- Dark mode, dashboard customize, lọc lịch sử nâng cao.
- Cho phép export PDF/Excel trực tiếp từ giao diện.

- **Bảo mật & vận hành:**

- Mã hoá end-to-end cho audio/video stream.
- Ứng dụng mô hình kiểm thử tự động giúp kiểm tra chất lượng sản phẩm.